claude-windows / e03c8ada-c18d-40e6-9ee1-144846fe3975

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e03c8ada-c18d-40e6-9ee1-144846fe3975.jsonl`
- SHA-256: `83a89ea265f5aa4ea2da128c6eb06d03bdd18f0cea095055474094642dd3f0dd`
- Source modified: `2026-06-22T10:15:25+00:00`
- Imported at: `2026-07-05T16:48:19+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `e03c8ada-c18d-40e6-9ee1-144846fe3975`

Transcript

1. user (2026-06-14T10:30:31.151Z)

Confirm the just-landed Gate-A default flip (PR #146, merged to master as 570cf2b) causes no served-side regression on the real fusion path.

Context: PR #146 flipped the served default to `fusion_hybrid` with `indexing.fusion.min_crossings=2` (Gate-A held-shuttle guard, net_crossings>=2). The flip is validated OFFLINE (LOVO ablation, n=6 golden: ?F1 +0.074, CI [+0.042,+0.105], 6/6, p=0.031) and unit-tested, but the production `FusionSegmenter` served path (full WASB-via-WSL + Gemini handoff) has only ever been mock-tested on real video ? its byte-identity to `wasb_hybrid` at `min_crossings=0` was verified by #117's review but never run end-to-end with Gemini.

Task: on the GPU machine (torch+CUDA, WSL Ubuntu WASB, Gemini key in .env all present), run ONE or a few golden videos through the live `fusion_hybrid` segmenter two ways ? `indexing.fusion.min_crossings=0` (control = ungated, should reproduce wasb_hybrid) and `min_crossings=2` (Gate-A default) ? and confirm: (a) the control run's served rallies match a `wasb_hybrid` run on the same video (byte-identical seeding/handoff at mc=0); (b) the Gate-A run drops only low-crossing (held-shuttle/feed) windows, no real-rally loss vs the golden labels (or at least no F1 regression). Use the existing offline-rescore tooling where possible to avoid re-running WASB (wasb_runner has NO cache-hit early-return ? re-running overwrites the cached CSV; re-score the cached `output/wasb/*_wasb.csv` instead, see scratch/run_gx010125_rescore.py and backend/eval/distill_local.build_candidates). Keep artifacts isolated (per-video output dir). Record the result in docs/QUALITY_ITERATIONS.md (honest measured numbers only). If a regression appears, the rollback is a single flag: set `indexing.fusion.min_crossings: 0` (Gate-A off) or `indexing.default_segmenter: wasb_hybrid` (full revert) in config.json.

This is the optional owner-gated follow-up flagged in PR #146 ? it does NOT change the merge decision; it adds served-side confidence.

2. assistant (2026-06-14T10:30:43.124Z)

I'll start by orienting myself: pulling the repo, reading the current-state/handoff, and checking what's actually available on this machine (the task assumes a GPU machine with torch+CUDA+WSL+Gemini).

3. user (2026-06-14T10:30:47.126Z)

```
Already up to date.
---BRANCH---
master
---HEAD---
e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
570cf2b feat(fusion): flip Gate-A (held-shuttle guard) to the served default ? config-reversible (#146)
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)
```

4. user (2026-06-14T10:30:47.146Z)

```
---
name: session-handoff
description: "Living \"you are here\" handoff ? current state, next steps, open decisions, ramp-up kit. AUTO-REFRESHED at meaningful checkpoints (PR merged, key decision, phase shift, session wrap). \"update the handoff\" forces a full rewrite."
metadata:
  node_type: memory
  type: project
  originSessionId: 45ea3a91-654d-4972-9cd3-65f35db54caf
---

# Session handoff ? sports highlight indexer (repo still named badminton-highlight-indexer; RENAME PENDING) + siblings: sports-data-collector, rally-annotator, sports-obsreport

**Last updated:** 2026-06-14 (session wrap; master at #135; self-merged on green CI throughout). **?? IMMEDIATE PICKUP =
EXECUTION-POLICY P3 (then P4)** ? a per-job, self-scheduling resilience framework (owner's design, validated). **DESIGN
([#132](https://github.com/avidullu/badminton-highlight-indexer/pull/132) `docs/EXECUTION_POLICY.md`) + P1 the actual
Gemini-hang fix ([#133](https://github.com/avidullu/badminton-highlight-indexer/pull/133)
`backend/infrastructure/execution_policy.py` ? `run_bounded`/`poll_until`, 100% module coverage) + P2 the self-scheduling
DB substrate ([#135](https://github.com/avidullu/badminton-highlight-indexer/pull/135) ? jobs migration v3 `policy`/`next_poll_at`,
`claim_due_job`/`set_job_waiting`) are MERGED.** **NOT DONE (next session, own distinct PRs, HIGH coverage):** **P3** =
wire the `main.py` executor to a `claim_due_job` self-scheduling loop honoring WAIT_AND_RESUME (release+reschedule) +
**resumable AI handoff** (persist per-window confirmation; resume only unconfirmed ? the GX010125 reel becomes exhaustive
across resumes, no wasted Gemini spend); **P4** = apply the policy to the WSL/WASB GPU pass + ffmpeg. **Read [[current-state]]
FIRST** (the top checkpoint has the full P1/P2 detail + the rationale for stopping at P2).

**Earlier this same session (rally-quality / paper track, [[paper-coauthor-aim]]):** analyzers/reconcilers **framework**
doc + a verified **multi-agent literature review** (§13 ? skeleton is standard TAL/TAS+late-fusion, *cite don't claim*;
only the report-first reconciler is a novel combination) ? [#119]; **eval measurement-honesty helpers C1?C4** (`backend/eval/
eval_stats.py`+`score_calibration.py`+`segmentation_metrics.py`) ? [#124] + wired into `experiment.compare` ? [#129];
**FIRST golden A/B experiment** (n=6, offline) ? archived [#128]: **Gate-A (net-crossings?2) is a measured win (+19% F1,
6/6, p=0.031), PROMOTED** [#131]; the window-level **learned variant FIXED** (train-fold threshold tuning + calibration ?
+38% F1, un-zeroed) ? [#130]; **`docs/PER_VIDEO_OUTPUT_LAYOUT.md`** plan [#131]; **GX010125_1080p reel re-generated** into
isolated `output/GX010125_run/` (reused cached 1080p trajectory). Prior tracks still valid: rally-quality fusion P1?P4
(#114?127), UI-alpha, productization, first 4K-GPU E2E. This is the ~1-page orientation. ? **THIS IS THE OWNER'S REAL GPU
MACHINE** (torch+CUDA/cv2/WSL/Gemini live).

## You are here (2026-06-13) ? ? HARDENED + M0 SCAFFOLDED + MOAT QUANTIFIED + QUALITY-SWEPT + UI-ALPHA LANDING + 4K-E2E PROVEN
-1. **Recent landings since the sweep (mostly OTHER slates ? facts from git/gh, not this session's work):**
   - **UI-alpha:** PR-1 async jobs (#87) + **PR-2 chunked upload + highlight download (#99, live-E2E-verified)** +
```

docs sync (#100). Next UI slice = **PR-3 identity** (`owner_email` scoping, JWT/header w/ `DEV_USER_EMAIL` fallback, CORS narrow, rate limit). ? PR-3/deploy: `security.allowed_video_root` MUST contain `upload_dir`.
- **Branding/stitcher:** #101 configurable watermark · #102 config paddings ? live VideoCompiler · #103 bundled Apache-2.0 font + fallback log · #107 `/api/compile` enforces `stitching.min_shot_count` · #108 escape Windows drive-letter colon in ffmpeg filtergraph (fixed #103's watermark silently failing on Windows).
- **PMF/docs:** #104 C13 field kit (flyer/playbook/answer-sheet/pre-registered signals; associate-pay REMOVED per owner) · #105 i18n plan (Kannada-first) · **OPEN #106** video-locality model (timestamps are the product, video is dead weight) ? awaiting owner.
- **First 4K GPU E2E (2026-06-13):** real 11.5GB GoPro match ? `output\GX010125_highlights.mp4`, 11 rallies (>7 shots) of 52 detected; 1080p-proxy-index / 4K-stitch recipe in [[gotcha-long-runs-gpu-wsl]]. Suite 409 green.
- ? **IN-FLIGHT, uncommitted (sibling/owner ? do NOT clobber, not yet PR'd):** WSL **cache-hygiene** slice on `WslCommandMixin` ? `_wsl_free_gb`/`_wsl_rm_rf` + `keep_frames`/`min_free_gb` knobs + disk guard + post-success cache delete (base/wasb_runner/tracknet_runner/models.py); `config.json` GPU knobs; `arial.ttf`; `docs/STORAGE_SHARING_MODEL.md`.

0. **8?9 quality sweep DONE (2026-06-11, this session's prior work):** lint/type/coverage gates live in CI (ruff.toml, mypy.ini w/ quarantine ratchet, --cov-fail-under=70); trajectory hybrids share one engine (`trajectory_hybrid.py`, equivalence-tested); **wasb_hybrid's FIRST verified live E2E run** found+fixed two real live-path bugs (#97 WSL tilde+abspath); LOVO 0.626 re-verified EXACT post-sweep. Merged #90?#98. Deferred: ruff I/UP, yolo_hybrid fold-in, pyproject (owner deselected), extra='forbid'.
1. **Audit + remediation DONE:** 8-auditor deep audit (all findings adversarially verified) ? 12-PR hardening sweep (both repos). Master was literally broken (compiler SyntaxError) and is now CI-guarded (both repos have CI; merges gated on verified-green). Corpus licensing corrected (23 ShuttleSet broadcast rows demoted; commercial export = owned data only). Suite: indexer 336 green, collector 136 green.
2. **ADR-008 ratified (owner):** training in-repo (`training/`) behind a CI-enforced import wall; featurizer single-sourced in `backend/features/rally_seq.py` (train==serve); serving consumes versioned ARTIFACT BUNDLES only; ship-gate product-side; **pre-committed repo split** (`sports-temporal-models`) at Gen-2/second-consumer. "Split driven by productionization, not isolation" (owner's framing).
3. **M0 executed:** canonical GT loader (`backend/eval/gt_loader.py` ? closed the two-loader format fork, round-trip tested vs collector export) · **n=6 owned corpus IN the collector SoR** (262 rallies, Proprietary) · **Gen-0 harness** `python -m training.gen0.harness` (train?LOVO?gate?artifact; v0.1.0 built: **LOVO 0.626**, floor 0.480 passed, sign test 5/6 p=0.109 ? ship=False with honest blockers).
4. **Gemini battery DONE (the moat is quantified):** clean-ish Mahadevpura ? heuristic 0.657 / Gen-0 0.744 / two-stage refine 0.83 / **wrapper 0.93**; multi-court Kushagra ? heuristic 0.157 / Gen-0 0.561 / **wrapper 0.66** (FPs = background-game pickups + dead-time merges, the contrast-metric confound). **Ship target = beat 0.66 on multi-court**; clean footage is commoditized (serve via Gemini). Baselines committed in `eval_baselines/`. Demo reel exists: `output/MahadevpuraSingles_highlights.mp4`.

5. **Gemini ops (hard-won):** prepaid/cold projects burst-throttle with a MISLEADING "credits depleted" 429 ? ramp gradually; serial works (`ai_max_workers=1` shipped); chunks re-encode to 1280 (`ai_chunk_scale_width`, PR #80); generate retries = 6 attempts exponential+jitter (PR #83); model = `gemini-flash-latest` alias (2.5-flash is legacy). 503 capacity waves are real ? provider-fallback registry is load-bearing.

Owned-model architecture (for clarity ? owner asked)
Gen-0/Gen-1 = WASB-substrate + OWNED temporal head. WASB (frozen, MIT code, C3 checkpoint provenance = ship blocker) produces trajectories; the owned IP = featurizer + head + corpus + harness/gate. **Gen-2 (Qwen3-VL via ms-swift) = pixels, drops WASB** ? that's the heavyweight step, only entered if the lightweight head plateaus below target. Gen-0 trains in MINUTES on the 2070S ($0). Multisport: WASB transfers to TT (0.909) but not pickleball (0.308) ? per-sport detectors or Gen-2.

?? Next steps / open owner decisions (mirror of docs/NEXT_STEPS.md ? read that for detail)
0. **? RESUME HERE ? EXECUTION-POLICY P3 then P4** (`docs/EXECUTION_POLICY.md` §Phasing). **P3** (one PR): in `backend/main.py`, replace the fire-once `_job_executor.submit(_run_job)` with a loop that calls `db.claim_due_job()` (already built, #135) and, when a long op hits its policy budget, calls `db.set_job_waiting(job_id, delay_sec)` to release+reschedule (WAIT_AND_RESUME) instead of blocking; **+ resumable AI handoff** ? persist per-window confirmation (candidate `stats` or a small table) so a resumed handoff only does the UNCONFIRMED windows (this is what makes the GX010125 reel exhaustive across resumes, no re-spend). Tests: a fake handoff that hangs ? job goes `waiting` ? re-claimed ? resumes only the unconfirmed window. **P4** (one PR): wrap the WSL/WASB `_wsl_bash` GPU pass + the ffmpeg calls in `run_bounded`/`poll_until`. **Each = distinct self-merged PR, HIGH coverage** (owner's explicit bar). Substrate (P1 `execution_policy.py` + P2 `claim_due_job`/`set_job_waiting`) is done, so P3 is wiring not invention.
1. **Owner mission in progress: grow the golden corpus** (multi-court badminton first; ?3 matches/sport, TT next; adults-only for training; per video: label ? `import-local --normalize` ? re-run harness). Sign test reaches significance around n=10 with 9 wins.
2. **Set the ship-gate target** (suggested: LOVO F1@tIoU0.5 ? 0.70 on multi-court folds + paired-sign p<0.05).
3. **M0.4 increment:** ActionFormer/ASFormer into the harness (still lightweight/local) + per-video threshold calibration.
4. C13 interviews + camera pilot (PMF) ? **field kit SHIPPED (#104):** flyer/visit-playbook/answer-sheet/pre-registered signals (associate-pay removed per owner; he handles engagement terms); repo rename; rotate the chat-pasted Gemini key; collector=SoR confirm.
5. **UI-alpha track (owner-gated):** PR-1 async jobs (#87) + PR-2 upload/download (#99) DONE ? **next = PR-3 identity** (`owner_email` scoping, JWT/header auth w/ `DEV_USER_EMAIL` dev fallback, CORS narrow, rate limit, quota). The PR2_HANDOFF "tomorrow-morning CUJ" runbook is unblocked for the owner.
6. **Open owner decisions:** ratify/merge **#106** video-locality model (it reframes hosting ? timestamps are the product); whether the in-flight WSL cache-hygiene slice gets PR'd by its owner or folded into the next run.

Ramp-up kit

112   - [[current-state]] FIRST (dense checkpoints) · `docs/NEXT_STEPS.md` (live plan) · `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`
113   (roadmap) · ADR-008 in `docs/archives/decisions/DECISIONS.md` (topology) · `docs/CODE_MAP.md` (note its 2026-06-10
114   sweep banner ? several documented behaviors changed).
115   - Code anchors: `training/gen0/harness.py` · `backend/features/rally_seq.py` · `backend/eval/gt_loader.py` ·
116   `eval_baselines/*.json` · collector `src/cli/curate.py` (reclassify-licenses, import-local --normalize/--force).
117   - **This session's anchors:** `docs/EXECUTION_POLICY.md` + `backend/infrastructure/execution_policy.py` (P1) +
118   `database.py::claim_due_job`/`set_job_waiting` (P2) + `backend/main.py::_run_job`/`_job_executor` (the P3 target) ·
119   `docs/ANALYZERS_AND_RECONCILERS.md` (§13 = the verified prior-art/course-corrections review) · `backend/eval/`
120   {`experiment.py` (now emits a `honest` block: CIs + sign test + F1@k by default; `Variant.tune_threshold`/`calibrate`),
121   `eval_stats.py`, `score_calibration.py`, `segmentation_metrics.py`} · `docs/archives/quality-iterations/2026-06-first-ab-experiment/`
122   (the first A/B: Gate-A +19% F1) · `scratch/{run_gx010125_full,run_gx010125_rescore,ab_n6_lovo}.py` (machine-side repros).
123   - Data: labels `Annotation Setup\Collect\Golden Labelled\` · trajectories+Roora `...\Collect\Trajectories\` ·
124   eval manifest `output/human_lovo_manifest.json` (gitignored) · corpus DB = collector `data/sports_metadata.db`
125   (backup: `data/sports_metadata.backup-2026-06-10.db`, gitignored).
126
127   ## Gotchas (this session's additions)
128   - Merge discipline: gate on `gh pr view --json statusCheckRollup` == SUCCESS (a piped `gh pr checks --watch` exit code
129   once let a red PR merge ? #75/#76 incident). Collector DB: stash-and-pop, never `checkout --` (recovered once from
130   a dangling stash). Other sessions may write [[current-state]] concurrently ? insert checkpoints, never overwrite.
131   - PowerShell `ConvertTo-Json` reformats config.json wholesale ? edit surgically. `Select-Object -Last N` buffers
132   background output until exit (no interim tail) ? and loses EVERYTHING if the process is killed (idle/suspend kills
133   background runs silently); pipe long background runs through `Out-File` instead.
134   - **Sibling sessions share this checkout and flip branches mid-session** (the node24 chip did) ? re-verify
135   `git branch --show-current` before EVERY commit; the post-merge "checkout master && pull" scripting once left the
136   tree on master and a commit went straight to the default branch (the AI-7 process slip ? don't repeat it).
137   - Windows ffmpeg here lacks NVENC ("No capable devices found") ? use libx264 for host-side re-encodes.
138   - The dev container claim in CLAUDE.md is stale for THIS machine: cv2/torch/numpy all present; GPU via WSL.
139

5. user (2026-06-14T10:30:47.528Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-451 of 1851 total (87024 tokens, cap 25000). Call Read with offset=452 limit=451 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

1   ---
2   name: current-state
3   description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not only on request). What just landed, what's next, open decisions."
4   metadata:
5   node_type: memory
6   type: project

originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301

Current state ? updated continuously (see [[working-agreement-checkpoints]])

Checkpoint: 2026-06-14 (PERSONALIZATION Phase-0 LANDED + quality checkpoint). **Owner adopted the HYBRID** ? personalization as a flag-gated FEATURE on a generalization-first baseline + a **contribution flywheel** ("a tool that gets better the more you help it"; free-tier videos pool into the owner-trained baseline). Pivot analysis (`scratch/PERSONALIZATION_PIVOT_ANALYSIS.md`) recommended hybrid-not-personalization-first; the owner's contribution-flywheel closes the flywheel-loss objection. **MERGED this session:** telemetry wire-in **#140**; per-user promotion gate **#141** (min-N floor + structural-guard exemption); held-out-USER learning curve **#142** ("<K videos to superior quality" = falsifiable); per-user no-regression guard **#143**; **`docs/PERSONALIZATION_PLAN.md` #144** (the checked-in project doc + locked decisions). Gemini-training-source concern = **CLEAN** (eval-only, gitignored, M0.3 owner-gated). **LOCKED owner decisions:** identity = north-star-design/current-impl (nullable user_id, device UUID); inline classifier_json behind a cloud-load interface; **`min_n` is a PROMOTION-confidence floor, NOT a service gate** (a 1-video user is always fully segmented by the baseline); free-tier video pooling (consent + minors-exclusion); **Gate-A-only structural + `min_n=5`**. ?? **IN FLIGHT (3 bg agents):** quality-iterations doc PR; **Gate-A/B golden-set numbers + flip-implications** (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` ? leave-one-signal-out ablation; owner: NUMBERS INFORM, FOUNDATIONS DECIDE); **ablation-layer framework plan** (`scratch/ABLATION_FRAMEWORK_PLAN.md` ? leave-one-signal-out + signal-relevance ledger over time, for the paper). **GPU + Gemini API now AUTHORIZED for evals** (cooler boost on). **Gate-A default flip = NOT done** ? gather numbers + implications first, then owner decides (data-influenced, foundations-led). NEXT personalization = Phase 1 (serving bridge: wire `LogisticRegression.load` into `_select_for_handoff`) ? owner-gated, see [[weak-signals-retained]] + `docs/PERSONALIZATION_PLAN.md`.

Checkpoint: 2026-06-14 (FUSION P3/P4 MEASURED + extraction COMPLETE + telemetry #134 merged + Cooler Boost). **THE NUMBERS (honest LOVO n=6, IoU?0.5, C1 bootstrap-CI + sign-test):** shuttle-only F1=**0.307**; **+person (P3) ?F1=?0.021** (?6.7%, CI [?0.165,+0.162], sign 2W/4L p=0.688, **CI-clears-0: NO**) ? NO lift, stays default-OFF (no court polygons ? `players_in_court` counts spectators; per-video wash: testlarge +0.40 / gbaaddy ?0.22). **+person+audio (P4 incremental) ?F1=+0.079** (+27.7%, F1 0.286?**0.366**, CI [?0.054,+0.203], sign 4W/2L p=0.688, **CI-clears-0: NO**) ? PROMISING (wins 4/6, largetest/mahadevpura +0.27; endpoint 0.366 > shuttle-only 0.307) but n=6 can't confirm. **VERDICT: neither promotable (both CIs span 0); audio is the lead ? need more matches + measure audio on shuttle-only directly (drop the person handicap).** Recorded in `eval_baselines/experiment_leaderboard.json`; honest negative/uncertain results KEPT ([[paper-coauthor-aim]]). **LANDED THIS SESSION:** result logged to `docs/QUALITY_ITERATIONS.md` (#137); `experiment_leaderboard.json` tracked (#139); **weak-signals audit ?** (person/audio/court-bounds all present, default-OFF, no-special-casing, retained ? refuted=false ? [[weak-signals-retained]]); 7-page paper-style **quality report PDF** at `OneDrive\Documents\Rally_Detection_Quality_Report.pdf` (gen `scratch/quality_report.py`; conservative academic restyle, strict-reviewer-approved); `FusionGoldenExtract` scheduled task **unregistered** (cleanup); tree clean on `master`. ?? **NEXT (fusion track):** (1) **wire `run_telemetry` (#134) into fusion_golden + the ps1 wrapper** (deferred, ready, mergeable); (2) **measure audio on shuttle-only directly** (quick eval, reuses the 6 `output/*_golden_features.json`, NO new data ? audio's standalone lift without the person handicap); (3) **grow the golden corpus via Roora** (owner's job ? gives power to confirm the +0.079 audio lead) + **activate court polygons** (re-measure person properly ? the ?0.021 is partly a `court_polygon=None` artifact); (4) future **shuttle-dropped** cue ([[weak-signals-retained]]). **OTHER track (per [[session-handoff]] ? the stated IMMEDIATE PICKUP): EXECUTION-POLICY P3 then P4** (owner-gated; #132/#133/#135 done). **Extraction infra** (all working): detached Scheduled Task `FusionGoldenExtract` runs `scratch/extract_until_done.ps1` (self-restart loop + in-process keep-awake + GPU-busy gate: waits if free VRAM<3500MB) ? deaths were `0xC000013A` console-control kills (NOT crash/sleep/TDR/OOM ? verified via event log), auto-recover via 3-min repetition trigger + restart-on-failure. **Cooler Boost** (MSI Creator Center) dropped 85?67°C, SM ~1100?~1300MHz, velocity ~26?~50 fps (the rest is Max-Q power cap, not thermal).

Checkpoint: 2026-06-14 (EXECUTION-POLICY framework: design + P1 + P2 SHIPPED ? **PR #132 (design) + #133 (P1) + #135 (P2) MERGED**, self-merged on green CI). Owner asked to harden the GX010125 Gemini hang as a **policy-driven, per-job, self-scheduling** system (default WAIT_AND_RESUME, ABORT, POLL_EVERY_N_SEC?, **no master node**) with **very high coverage** (failures cause delays) and **clean poll-logging** (visible but not polluting). **#132** = design doc `docs/EXECUTION_POLICY.md` (validated the owner's architecture: per-job policy = data on the job; self-schedule via re-enqueue; no master ? rides the existing single-worker executor + `jobs` table + crash sweep; DB is the coordinator). **#133 (P1, the actual fix)** = `backend/infrastructure/execution_policy.py`: `ExecutionPolicy` + `run_bounded` (HARD `op_timeout_sec` per-attempt ? the hang-killer; retries failures AND hangs; hung attempt's daemon thread abandoned) + `poll_until` (bounded poll loop). Clocks injected ? **12 tests, 100% module coverage, instant**. Refactored `gemini.py`: `generate_content`?`run_bounded` (a never-returning generate previously blocked FOREVER = the GX010125 hang; now bounded), processing-wait?`poll_until` (**per-poll logs at DEBUG, not an INFO line every 10s** ? the no-pollution ask). 23 gemini tests stay green. **#135 (P2, substrate)** = DB migration **v3** (`jobs.policy` JSON + `next_poll_at`; existing rows survive via ALTER) + `set_job_waiting(delay_sec)` (self-schedule: park 'waiting' until due) + **`claim_due_job()`** (atomic compare-and-set claim of queued/due-waiting ? running; concurrent-safe, no master) + `create_job(policy=)`/`get_job` policy. **Additive, no executor change**; 10 tests; version asserts bumped 2?3. ? **THIS IS THE OWNER'S REAL GPU MACHINE** (torch+CUDA/cv2/WSL/Gemini live). ?? **REMAINING (owner-gated, NOT done ? deliberately not rushed at session tail given the high-coverage bar): P3** = wire the executor (main.py) to a `claim_due_job` self-scheduling loop honoring WAIT_AND_RESUME (release+reschedule) + **resumable AI handoff** (persist per-window confirmation; resume only unconfirmed ? the GX010125 reel becomes exhaustive across resumes, no wasted Gemini spend); **P4** = apply the policy to the WSL/WASB GPU pass + ffmpeg. Each = its own distinct PR. Ties [[paper-coauthor-aim]] indirectly (infra).

Checkpoint: 2026-06-14 (ACTED ON FIRST-A/B FINDINGS: Gate-A PROMOTED + learned FIXED + GX010125 run ? **PR #130 + #131 MERGED** to master `2e5892e?`; self-merged on green CI). ? **THIS IS THE OWNER'S REAL GPU MACHINE** (torch+CUDA, cv2, WSL Ubuntu, Gemini key in .env all present) ? not the dev container the older CLAUDE.md note assumes. **#130 = learned-variant FIX**: additive/default-OFF `Variant.tune_threshold` (operating point on the TRAIN fold) + `calibrate` (Platt/isotonic, C2); validated on n=6 golden ? **mean held-out F1 0.307?0.423 (+0.116), testlarge 0.000?0.300

(un-zeroed), now BEATS CONTROL 0.388**; threshold-in-LOVO is the big lever. **#131 = Gate-A PROMOTED** (first cue promoted via the harness; doc trail in QUALITY_ITERATIONS ? measured win dF1 +0.074, 6/6, p=0.031; recommended `indexing.fusion.min_crossings=2`; **live served-default flip stays OWNER-GATED** until a real fusion-path run confirms ? fusion P2 is unverified on real video) **+ new `docs/PER_VIDEO_OUTPUT_LAYOUT.md`** (one dir per video; fixes the hardcoded `"output"` joins in `trajectory_hybrid.py:~176` so `--output-dir` truly isolates). **GX010125_1080p RUN (offline, no GPU):** ? key finding ? `wasb_runner.run_predict` has **NO cache-hit early-return**, so the full pipeline would RE-RUN WASB (~tens of min on 1080p) + overwrite the cached CSV; the cached trajectory is reusable ONLY via offline re-score. So ran `scratch/run_gx010125_rescore.py` = parse cached `output/wasb/GX010125_1080p__89605e80ed01_wasb.csv` ? 45 candidates ? CONTROL 45 rallies (11.0min) vs **Gate-A 27** (9.3min), Gate-A **dropped 18 low-crossing windows (~1.7min)** of likely held-shuttle/feed FPs ? isolated `output/GX010125_run/` (no pollution; no golden labels for this footage ? divergence not F1). A FULL Gemini-confirmed highlight reel = a separate ~tens-of-min GPU+API run (owner kicks off). Repro: `scratch/{run_gx010125_rescore,ab_n6_lovo,ab_eval_demo}.py`. All in isolated worktrees off master (siblings concurrent). ?? NEXT (owner-gated): real fusion-path run to confirm Gate-A served-side + flip the live default; impl `PER_VIDEO_OUTPUT_LAYOUT` L1; Tier-2/3 §13 corrections; analyzers/reconcilers A1?A5/R1?R3. Ties [[paper-coauthor-aim]].

Checkpoint: 2026-06-14 (FIRST GOLDEN A/B EXPERIMENT + HARNESS HONEST-REPORTING WIRED ? **PR #128 + #129 MERGED** to master `f680b8d`, self-merged on green CI per [[working-agreement-merging]]; owner-directed momentum). Ran the **first real end-to-end A/B on the n=6 golden corpus, 100% OFFLINE** (re-scored persisted WASB trajectories via `distill_local.build_candidates`, no GPU ? the "persist once re-score" thesis proven). One candidate set re-scored three ways: **Gate-A (net_crossings?2) is a MEASURED WIN ? ?F1 +0.074, 6/6 matches, sign-test p=0.031, CI [+0.042,+0.104], seg-count-ratio 1.68?1.01** (over-seg nearly eliminated); the **window-level 5-feature logistic did NOT clear the bar** (?0.081, 2W/4L; testlarge zeroed = threshold/calibration artifact ? C2 motivation; **distinct from the stronger per-frame `rally_seq_proto` 0.583** ? don't conflate). Headline: **the framework held up ? C1 promoted one variant and refused another (right call both ways); C4 surfaced over-seg tIoU hid.** **#128** = archived posterity doc `docs/archives/quality-iterations/2026-06-first-ab-experiment/` (full tables) + archives-index + live QUALITY_ITERATIONS pointer. **#129 = (b) wired C1+C4 into the CORE `experiment.compare()`** (additive/default-ON `honest_stats=True`: `honest` block = per-metric bootstrap CIs + paired ?F1 CI + exact sign test + F1@k + segment_count_ratio; `record_experiment` carries honest ?F1; existing fields byte-unchanged, `honest_stats=False` = legacy shape). Reuses #124 helpers; complements sibling **#127** which wired the same C1 into `fusion_compare` (no dup). 19 existing experiment tests pass + 3 new; ruff+mypy+CI all green. Repro `scratch/ab_n6_lovo.py` + `scratch/ab_eval_demo.py` (machine-side; read gitignored manifest/external golden). All done in **isolated worktrees off master** (siblings merged #121-#127 concurrently). ?? NEXT (owner-gated): promote Gate-A via the no-regression gate (it's a real measured win); fix the logistic (C2 calibration + threshold-inside-LOVO); Tier-2/3 §13 corrections; analyzers/reconcilers impl A1?A5/R1?R3 (none started). Ties [[paper-coauthor-aim]].

109 **Checkpoint:** 2026-06-14 (FUSION P3/P4 MEASUREMENT BUILT + GPU EXTRACTION RUNNING ? master `d7b1f51`; suite 514 green).

110 The "modeling" track for the person-cue (P3) + audio (P4) fusion lift ? all default-OFF, no-special-casing, harness-measured.

111 **Merged this run (mine): #121** fusion P3 (`backend/eval/fusion_golden.py` offline person-feature EXTRACTOR + `fusion_compare.py`

112 LOVO driver ? reuses build_candidates/PersonDetector/fusion_features/experiment.compare, NO new scoring); **#122** P4

113 (`backend/pipeline/detectors/audio_features.py` + `backend/eval/fusion_audio.py` incremental audio cue); **#123** progress-logging

114 + resumability (`--fresh`); **#125** single-pass `PersonDetector.detections_over_windows` (fixed the per-window-seek O(N)

115 pathology ? ~28s/window ? one sequential decode); **#126** `--smallest-first`; **#127** honest ?F1 = bootstrap CI + paired

116 sign test on per-video held-out F1 (wires #124's `eval_stats.paired_delta_ci`; "never a bare mean"). Concurrent sessions

117 merged #119 (analyzers framework), #120 (run.log), #124 (eval-honesty utils: eval_stats/score_calibration/segmentation_metrics).

118 **? GPU EXTRACTION via DETACHED SCHEDULED TASK `FusionGoldenExtract`** (runs `scratch/extract_until_done.ps1` ? Task Scheduler

119 owns it, OUTSIDE Claude's job object ? survives turn/context/session boundaries; in-process keep-awake + self-restart loop until

120 6/6; clean UTF-8 logs to `output/fusion_golden.log` + `scratch/extract_wrapper.log`). Loops `python -u -m backend.eval.fusion_golden

121 --manifest output/human_lovo_manifest.json --out-dir output --smallest-first` ? 6 `output/<name>_golden_features.json` (GITIGNORED

122 + RESUMABLE: skips any video whose JSON exists). **3/6 DONE** (testlarge, mahadevpura, largetest); remaining

123 kushagra?gbaaddy?adarsh(last, 84k frames). **MONITOR:** JSON count + `tail output/fusion_golden.log` + `nvidia-smi`;

124 `Get-ScheduledTask FusionGoldenExtract|Select State`. **RESTART (resumes, skips done):** `Start-ScheduledTask -TaskName

125 FusionGoldenExtract`. **STOP:** `Stop-ScheduledTask -TaskName FusionGoldenExtract` + `Get-Process python|Stop-Process -Force`.

126 ?? **WHY DETACHED (debugged 2026-06-14):** Claude-bg-task runs DIED TWICE silently (largetest 61%, kushagra 6% @137s), NO Python

127 traceback AND ? verified via Windows event log ? NO sleep (Kernel-Power 42), NO TDR (Display 4101), NO WER crash, NO RADAR-OOM

128 (no event 1003; 18GB free). Traceless vanish = external `TerminateProcess`; common factor = both were Claude-managed bg tasks

129 (job-object kill-on-close at a context boundary fits death #1 @ the prior context summary). Scheduled Task = immune; GPU access

130 CONFIRMED in the interactive task session (53?64% util, 3.1GB). GPU util while running = 20?64% bursty ? batch=1 Faster-RCNN

131 latency-bound (small kernels + CPU NMS + per-frame ByteTrack), NORMAL, model IS on CUDA, NOT decode-bound; batching the forward

132 pass is the next-extraction speed lever (changes features, so NOT mid-run). ~1.5-2h total. Stale Claude bg keep-awakes stopped. **DEATH #3 + GPU-job check (2026-06-14):** died a 3RD time (gbaaddy 69%)

133 ? `LastTaskResult=0xC000013A` = STATUS_CONTROL_C_EXIT = terminated by a **console-control event / CTRL+C** (NOT crash/sleep/TDR/OOM

134 ? explains the no-trace vanish; killed the detached task too, so NOT Claude's job object). GPU-contention angle CHECKED: **NO

135 competing GPU compute job** (only desktop apps + our 1 python; 5GB free; HW is a **laptop** RTX 2070S **Max-Q**, MSI). Mitigations:

136 (1) task re-registered with **3-min repetition trigger + restart-on-failure (count 10, 1-min)** ? auto-recovers within minutes of

137 ANY death; (2) **GPU-busy gate in the ps1** (owner policy = *wait until free, auto-resume*): before each python launch, if free

138 VRAM <3500MB (another job has the GPU) it waits 30s + re-checks, then launches. **?? WHEN IT COMPLETES ? the remaining steps for

139 the NUMBERS:** (1) `python -m backend.eval.fusion_compare --features-dir output --record` ? honest P3 LOVO ?F1 (shuttle-only

140 vs +person) w/ CI+sign-test; (2) `python -m backend.eval.fusion_audio --augment --compare --record` ? P4 incremental audio

?F1; (3) log both to `docs/QUALITY_ITERATIONS.md` I3 (HONEST measured numbers only ? [[paper-coauthor-aim]]). Honest caveat: NO court polygons ? `players_in_court` counts spectators (3-7 on "single-court" Mahadevpura); robust signals = motion + `shuttle_player_colocation`. GPU = RTX 2070S, person detection on Windows torch cu124 (NO WSL). Branch protocol held through heavy concurrent merges (#119-127) ? [[gotcha-shared-checkout-branch-collisions]]. On master, clean. **STORAGE (this session, owner-approved "delete cache + compact WSL"):** deleted the stale **93GB GX010125 frame cache** (WSL `~/clips` 95?2.3GB; WSL fs used 152?60GB, 896GB free; C: 53?72GB free) ? kept the 2.2GB GX010125 proxy. The WSL `ext4.vhdx` was `--set-sparse true` + `fstrim`'d (946GB trimmed) but this WSL build (2.3.24) did NOT auto-deallocate on shutdown ? the file is still **~221GB allocated on C:**. **DEFERRED to post-extraction: the physical ~150GB C: reclaim via `wsl --shutdown` then elevated `diskpart` ? `select vdisk file="...\CanonicalGroupLimited.Ubuntu_79rhkp1fndgsc\LocalState\ext4.vhdx"` ? `compact vdisk`** (heavy C: I/O would fight the GPU run's video reads). [[gotcha-long-runs-gpu-wsl]] · [[cache-hygiene-policy]].

Checkpoint: 2026-06-14 (RALLY-DETECTION FRAMEWORK doc + LITERATURE REVIEW + eval-honesty code ? **PR #119 + #124 MERGED** to master via self-merge on green CI per [[working-agreement-merging]]; owner-directed). **PR #119** (`docs/ANALYZERS_AND_RECONCILERS.md`): reframed the analyzers/reconcilers design into a **multi-scope signal-fusion framework** (frame detectors / window analyzers / **session analyzers**; warm-up `SessionAnalyzer` + shuttle-state frame-detector folded in) + **§13 = a verified multi-agent literature-review** (8 patterns surveyed, ~40 citations adversarially checked, **ZERO fabrications**; author-line fixes applied). **Verdict (honest):** the skeleton is STANDARD prior art ? candidate-windows+scorer = Temporal Action Localization; reconciler-as-refinement = Temporal Action Segmentation post-proc; "signal not verdict" = late/score-level fusion + stacked generalization (~20 yrs old, already in racket sports). **Only the report-first reconciler-as-linter bundle rated `novel-combination`**; warm-up-as-soft-feature = thinnest gap. **Cite, don't claim**; terminology realigned (producer?labeling function, scorer?stacked meta-learner, LOVO?leave-one-group-out, tIoU?F1@k/mAP). Ties [[paper-coauthor-aim]] (§13 is the honest related-work backbone). **PR #124** implements the **Tier-1 measurement-honesty course corrections C1?C4** as PURE ADDITIVE / default-OFF helpers (no existing compare/LOVO/metrics behaviour changed): `backend/eval/eval_stats.py` (C1 `mean_with_ci`/ `bootstrap_ci`/`paired_sign_test`/`paired_delta_ci` ? never a bare LOVO mean at n?6; C3 `out_of_fold_predictions`/`grouped_folds` ? no stacking leakage), `score_calibration.py` (C2 Platt+isotonic+brier/ECE ? comparable cue confidences), `segmentation_metrics.py` (C4 F1@{10,25,50} + mAP@tIoU + segment_count_ratio ? make over-segmentation tIoU is blind to visible). 24 tests; ruff+mypy clean; all 4 CI lanes green (full-suite coverage floor held). Both PRs done in **isolated git worktrees** (off master) to dodge the recurring shared-checkout branch-flips (siblings merged #117 fusion-P2, #118, #120, #123 concurrently; **#116 closed ? its content had landed via #117**). ?? NEXT (owner-gated): **wire C1?C4 into the DEFAULT harness reporting** (`compare`/`run_regression.py` emit CIs + F1@k/edit-score instead of a bare mean ? changes reported numbers, so its own PR) + the Tier-2/3 corrections in §13 (global reconciler decode, cue-correlation modeling, temporal smoothing, external ShuttleSet eval). Analyzers/reconcilers IMPL (A1?A5/R1?R3) still owner-gated, none started.

Checkpoint: 2026-06-14 (RUN.LOG READABILITY for manual QA ? **PR #120 MERGED ? master `6076eef`**; owner asked ? self-merged on verified-green CI per [[working-agreement-merging]] (squash; remote+local branch deleted); was branch `feat/runlog-readability` off `35a50f3`; CODE, minimal/config-gated). Owner's manual-QA loop = run the tool on a video ? eyeball the rally CSV against `run.log`. Two readability changes: (1) **suppress HTTP chatter** ? new central helper `backend/utils/logging_setup.py::setup_logging(verbose_http=False)` raises `httpx/httpcore/urllib3/google_genai/google.genai/ google.auth/grpc` to WARNING (NOTSET to restore), gated by NEW knob `logging.verbose_http: false` (`LoggingConfig` on `AppConfig`); our `backend.*` loggers untouched; wired into `run_process_and_stitch.py` (at import + re-applied post-config-load). (2) **end-of-run rally summary** ? new pure helper `backend/tools/export.py::format_rally_summary` (next to `segments_to_csv`/`basic_stats`): header (total rallies + total play HH:MM:SS) + table `# start end dur(s) shot_count` from the produced play_segments so log?CSV line up row-for-row; handles metadata dict-or-JSON-string, missing shot_count?`-`; HH:MM:SS via local `_fmt_hms` matching the segmenters' `_fmt_ts`; logged as ONE block at end of `main()` (also on the all-filtered early-return). Tests: `test_logging_setup.py`, `test_rally_summary.py`, +`verbose_http` default/override in `test_config.py`. Suite **477 green**, ruff (`.`) + mypy (`backend training`) clean. ? Commit-msg first attempt mangled (PowerShell `@'?'@` here-string in the *Bash* tool ? literal `@`; amended via `-F file`) ? Bash tool ? PowerShell. **BRANCH-COLLISION CLEANUP (owner asked):** after merge, LOCAL checkout left clean ? on `master`==`origin/master`==`6076eef`, 0 ahead/behind; pruned 6 stale LOCAL branches mapped to MERGED/CLOSED PRs (#92/#104/#111/#106/#102 + #116-closed-redundant) + the stale remote-tracking refs; **kept** `docs/analyzers-framework` (#119 OPEN). Working tree = only the intentional untracked carryovers (`scratch/`, `artifacts/`). REMOTE still has 15 prunable branches (14 merged-PR + #116) ? **offered to prune, owner did NOT answer the question ? left untouched** (must KEEP `feat/khelacharya-shot-intelligence` #11 = deliberately-retained sibling layer per [[sibling-internship-repo]], + #119). ?? NEXT: owner does the machine-side check (run on a real video ? eyeball the quieter log + summary block; dev container can't run the GPU pipeline); remote-branch prune available on request.

Checkpoint: 2026-06-14 (ANALYZERS DOC ? FRAMEWORK + paper framing ? **PR #119 OPEN, owner-gated, do NOT merge**; branch `docs/analyzers-framework` off master `35a50f3`; docs-only). Owner fed two more ideas this session and asked to fold them in and treat the doc as a **reusable framework / research paper**, not a project patch (ties [[paper-coauthor-aim]]). Extended the on-master `docs/ANALYZERS_AND_RECONCILERS.md` (which had landed via #117) from "two seams" into a **multi-scope signal-fusion framework**: producers emit SIGNALS at **3 temporal scopes** ? frame detectors (`RallyCue`) / window analyzers / **session analyzers** ? all consumed as feature columns by the learned scorer (spine unchanged: **no special-casing**, signal ?scorer, never `if/else`). Folded in: (1) **warm-up/practice** as a new **`SessionAnalyzer`** tier (whole-timeline span ? an `in_warmup` membership feature the scorer DOWN-WEIGHTS, NOT a hard `if t<warmup_end: drop`; measurable on golden TODAY since warm-up is already labeled negative); (2) **shuttle-state** (`in_air`/`racket_contact`/`inactive`) placed correctly as a **frame-scope detector** (a MULTI_SIGNAL_FUSION_PLAN `RallyCue`, NOT a window analyzer) ? derive-from-existing-signals first (velocity+visibility+direction-reversals+audio thwack), dedicated per-frame model ONLY where WASB trajectory gaps bite, with the per-frame-label blocker flagged. Added **§10 generality & research positioning** (weakly-supervised temporal segmentation;

standard-vs-contributed) with an explicit **attribution honesty caveat**: NO fabricated citations/results, related-work + ablations + >1-sport still owed, n=6 = direction-check only ? the honest backbone the paper aim needs. Diff = `2-seam?framework` (310+/137?). ? Shared checkout had flipped to master mid-task (the sibling's collision lesson recurred) ? re-verified branch, branched fresh, normal commit (no in-flight WIP left to isolate; #117/#118 already merged so all cross-links resolve). PR #116 stays CLOSED (content on master via #117). ?? NEXT (owner-gated): review #119; impl tracks unchanged (fusion P3 LOVO, then analyzers/reconcilers A1?A5 / R1?R3 per the doc's phasing ? each ONE PR, default OFF, promote on measured LOVO lift).

Checkpoint: 2026-06-14 (FUSION P2 SHIPPED + collision cleanup + paper aim ? master `35a50f3`; suite 467 green; 0 open PRs). **PR #117 MERGED = `FusionSegmenter`** (fusion P2, registry `fusion_hybrid`, **default-OFF**): shuttle-primary (windows seed from WASB/TrackNet) + adjudicating cues ? `net_crossings` (Gate A, always persisted, GPU-free via rally_gate w/ a reused axis estimate) + optional person cue (Gate B) persisting the ~5 `fusion_features.py` features (pure-tested) via `PersonDetector.detections_per_frame` over each window's ORIGINAL-video frame range. Plugs in via **two IDENTITY hooks** on `trajectory_hybrid` (`_enrich_candidate_stats`/`_select_for_handoff`) ? wasb/tracknet byte-identical (adversarially verified). Served gating optional+default-OFF (`fusion.min_crossings`/`min_players`) ? gates the handoff while persisting the full SUPERSET (offline substrate intact). Court mask = optional `fusion.court_polygon`. **Design-first** (`fusion-p2-design` workflow) ? **31-agent adversarial review** (25 findings?19 confirmed; headline = NO-REGRESSION verified) ? fixes folded (frame-dim guards, round() frame idx, rally_gate `estimate=` reuse, honest torch docstrings, +6 tests). **PR #118 MERGED** = README index fix (QUALITY_ITERATIONS + EXPERIMENT_HARNESS P1-SHIPPED). **? SHARED-CHECKOUT COLLISION (lesson):** the spawn_task DESIGN session ran in THIS checkout, committed `0f673ff` (the analyzers design doc) + flipped the branch BETWEEN my branch-point check and my `checkout -b` ? my fusion branch rode on top ? **#117's squash absorbed the design doc**, so **`docs/ANALYZERS_AND_RECONCILERS.md` is on master via #117** and **PR #116 was CLOSED as redundant** (content intact on master; #116's stale "OPEN do-not-merge" checkpoint below is superseded). Re-verify `git branch` AND that no sibling commit landed since, before `checkout -b`. **Owner side-aim ratified: co-author a paper** "local, cheap & efficient rally detection via multiple clever ideas" ? [[paper-coauthor-aim]]; `docs/QUALITY_ITERATIONS.md` is the ablation-ready empirical backbone (honest measured results only). **Analyzers+reconcilers DESIGN on master (owner-gated, NOT started):** injectable per-window analyzers (e.g. net-hit service-fault) + a report-first reconciliation pass; spine = signals?learned-scorer, **no special-casing** (owner directive). ?? NEXT (owner-gated): **fusion P3** (person features ? `experiment.compare` LOVO lift on the 6 golden videos ? the eager-person-compute cleanup lands here) + measure the Gate-A held-shuttle drop on GPU; then analyzers/reconcilers impl; P4 audio. Earlier this session: PR #114 harness, #115 person-cue extraction (see checkpoints below).

Checkpoint: 2026-06-14 (DESIGN DOC: two extensibility seams ? **PR #116 OPEN, owner-gated, do NOT merge**; docs-only, branch `docs/analyzers-and-reconcilers` off master `df7d5df`). Produced `docs/ANALYZERS_AND_RECONCILERS.md` (367 lines, a peer to EXPERIMENT_HARNESS + MULTI_SIGNAL_FUSION_PLAN) + a 1-line `docs/README.md` cross-link. **Spine (owner directive): NO special-casing in the decision path** ? every analyzer/reconciler emits a SIGNAL

(value+confidence); the SCORING MODEL
242 learns to weight it (already true in miniature: `experiment.predict`/`_gate_mask` treat `net_crossings`/`players_in_court`
243 as feature columns). **Seam 1 = injectable `WindowAnalyzer`s (forward)**: a registry (mirrors `segmenter_registry`) of pure
244 per-window cues over the already-produced per-frame-aligned signals (WASB trajectory + `PersonDetector.detections_per_frame`
245 + `rally_gate` net); each emits an `AnalyzerResult` = confidence-scored features ? `candidate_segments.stats`, events ?
246 `events` table (buffer-on-candidate, flush-on-confirm), optional boundary hints; worked example = a **service-fault**
247 analyzer (shuttle halts in net region ? `service_fault` signal + `net_hit` event + END hint); injects at the FusionSegmenter
248 `_enrich_candidate_stats` hook, flag-gated default OFF. **Seam 2 = reconciliation pass (backward)**: a report-first "linter
249 for rally decisions" ? `ConsistencyRule`s (trailing-dead-time?truncate *(owner's misfire fix)*, no-net-crossing?drop,
250 over-merged?split, boundary-slack?tighten) flag decision-vs-evidence conflicts; **report-first/apply-optional, NEVER silent**;
251 expressible as a harness `Variant` transform so LOVO-measurable on the 6 golden videos BEFORE auto-apply; idempotent, fixed
252 precedence. ? **DESIGNED AGAINST the IN-FLIGHT fusion-P2 WIP** (uncommitted in the shared tree: `fusion_hybrid.py`,
253 `fusion_features.py`, `FusionConfig.cue_flags`, `person_detector.detections_per_frame`, `docs/QUALITY_ITERATIONS.md`) ? that
254 WIP was kept **byte-intact** (I isolated my single README line via git plumbing; tree is CRLF / autocrlf=true, restored
255 owner README to exact session-start bytes 3841B). **Merge-order:** the doc cross-links QUALITY_ITERATIONS.md + the fusion
256 hook, neither on master yet ? **merge #116 after (or with) fusion-P2** (noted in the PR body). ?? NEXT: owner reviews #116;
257 implementation is the phased A1?A3 / R1?R3 plan in the doc (each ONE PR off master, default OFF, promoted only on measured
258 LOVO lift) ? all owner-gated, none started. Ties [[candidate-segments-finetuning]] + the harness ([[working-agreement-merging]]).
259
260 **Checkpoint:** 2026-06-13 (RALLY-QUALITY TRACK: HARNESS + PERSON-CUE EXTRACTION SHIPPED ? **PR #114 + #115 MERGED**,
261 master `df7d5df`; suite 421?441 green; 0 open PRs). **PR #115 (fusion P1):** extracted the torchvision person detector
262 + ByteTrack + velocity windowing out of `yolo_hybrid` into `backend/pipeline/detectors/person_detector.py`
263 (`PersonDetector`, a reusable cue producer); `yolo_hybrid` now ORCHESTRATES only (chunk/prefetch/AI-handoff/DB) and
264 delegates Stage-1 to `self.person`. **No behavior change** ? proven by the unchanged `process_video` output tests (mock
265 seam moved to `self.person`); pure detection unit tests ? new `tests/test_person_detector.py`. person_detector is
266 mypy-CLEAN (`self.model: Any`) so it needed NO quarantine; the torchvision code that put yolo_hybrid in quarantine left
267 it, but yolo_hybrid STAYS quarantined for PRE-EXISTING orchestration debt (implicit-Optional process_video defaults,
268 Success|Failure ABC-override mismatch, a Future[...] annotation ? flagged in mypy.ini to ratchet separately). ruff+mypy
269 clean, CI 4/4. Docs synced (fusion-plan P1?DONE, CODE_MAP person_detector entry + yolo_hybrid-as-orchestration, NEXT_STEPS
270 item 2 ticked). ?? **NEXT (owner-steered):** **PR 3 = fusion P2.** The cheap, highest-confidence, PURE-LOGIC piece =
271 **wire Gate A (`min_crossings`) into the live trajectory path** (`trajectory_hybrid`/`wasb_hybrid`) + add the knob to
272 `IndexingConfig` ? *fixes the owner's held-shuttle false positive*, default OFF, immediately A/B-able via the harness on
273 the 6 golden trajectories. Gate B (court-mask + player-count) is **owner-gated** on the court-mask-source decision
274 (MULTI_SIGNAL_FUSION_PLAN §9.1: manual polygon vs auto homography ? owner). **P3 learned

fusion + the SxS-on-new-videos RUN need GPU/owner-side** (persist person features on real video ? train on the 6 golden ? `compare_unlabeled(CONTROL, +person)` on new footage). Two PRs this session within [[working-agreement-merging]] self-merge authority.

Checkpoint: 2026-06-13 (EXPERIMENT HARNESS P1 SHIPPED ? **PR #114 MERGED**, master `ecee786`; suite 421?440 green). Picked up the rally-quality track (the desktop-agent pick-up `NEXT_STEPS` flagged; both design docs `EXPERIMENT_HARNESS.md` + `MULTI_SIGNAL_FUSION_PLAN.md` arrived via the session-start `git pull` 15e4e0e?9231af2, PR #112). Built `backend/eval/experiment.py` = the thin variant/experiment layer from `EXPERIMENT_HARNESS.md` P1 (the audit's "~80% already exists" HELD ? verified file:line; pure glue over metrics/calibration/classifier/training/regression.gt_hash/ gemini_refine.merge_overlaps/query_candidate_segments, **NO new scoring math**): `Variant` (JSON recipe: segmenter+config_overrides+cue_flags+Gate-A `min_crossings`+Gate-B `min_players`+learned classifier; `spec_hash`; pinned `CONTROL`), `compare` (labeled A/B, **LOVO-honest** train-on-others for learned variants, B?A deltas + `label_set_hash`), **`compare_unlabeled`** (the owner's **SxS-on-NEW-videos** ask ? agreed/a_only/b_only via match_intervals, no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), `load_video_candidates` (DB bridge). 19 tests incl. the load-bearing **no-regression invariant** (all cues OFF ? byte-identical to CONTROL); ruff+mypy clean; CI 4/4. Docs synced (EXPERIMENT_HARNESS P1?DONE, CODE_MAP entry + stale regression-baseline-path fix, NEXT_STEPS item 3 ticked). Owner answered ramp-up Qs: **corpus = all 6** golden videos (he said "5"; LargeTestVideo/TestLargeVideo may be cuts of one match ? kept all 6 per the manifest); **merge authority = self-merge on green then continue to PR 2** (within [[working-agreement-merging]]). ?? NEXT (this session): **PR 2 = fusion P1** person/vision-cue extraction (lift torchvision+ByteTrack+velocity out of `yolo_hybrid` ? `backend/pipeline/detectors/person_detector.py` producer; yolo_hybrid consumes it, no behavior change ? regression-safe). Then P3 learned fusion trains on the 6 golden videos + `compare_unlabeled(CONTROL,+person)` SxS on new footage (GPU/owner-side). Tracks [[candidate-segments-finetuning]] + [[collector-indexer-bridge]].

Checkpoint: 2026-06-13 (WORKING-TREE CLEANUP ? master `15e4e0e`, tree now CLEAN). Cleaned up the leftover uncommitted bits (the cache-hygiene CODE was already MERGED as #109/#110 per the checkpoint below ? nothing to rescue there): (1) `config.json` GPU-run knobs **reverted to committed defaults** (was wasb_hybrid/7200/8 ? gemini/1800/1; values stay recorded in the cache-hygiene checkpoint below ? re-apply `default_segmenter: wasb_hybrid` in one line if that's the intended new default; user didn't answer keep-vs-revert, took the safe recoverable path). (2) stray root `arial.ttf` (unused ? shipped watermark font is committed `backend/assets/fonts/RobotoSlab-Regular.ttf` from #103) ? moved to `scratch/`. (3) genuine untracked draft `docs/STORAGE_SHARING_MODEL.md` (190-line storage-anchored monetization model, owner co-design pending) ? branched + **PR #111 OPEN (owner-gated, docs-only ? do NOT auto-merge**; supersedes COMMERCIALIZATION.md framing if ratified ? ADR then). Master tree now shows ONLY intentional carryovers (untracked `scratch/` handoffs+repro scripts, `artifacts/` Gen-0 bundle) ? could be gitignored for a pristine `git status` if desired (offered, not done). Reminder: `python -m tools.cleanup_caches --apply` reclaims the 96.5 GB the SessionStart hook flagged.

Checkpoint: 2026-06-13 (CACHE HYGIENE shipped ? **PR #109 + docs #110 MERGED**, master `15e4e0e`; + the ?5 reel + manual cleanup). **#110** = diligence docs follow-up (README §Cache Hygiene + knobs + tools/ line; CODE_MAP detector self-clean, tools/cleanup_caches.py entry, common-task rows; min_shot_count API-parity note) ? docs-only, CI 4/4 green. Both repos now have **0 open PRs**. After the GX010125 E2E run left ~163 GB in WSL `~/clips`, built a full cache-hygiene system (all on master): (1) **self-cleaning runners** ? `indexing.{wasb,tracknet}.keep_frames` (default false) deletes the frame cache + staged video after a SUCCESSFUL run; on failure keeps them for resume; (2) **tools/cleanup_caches.py** dry-run-first ops tool (`--apply`/`--keep-video`/`--older-than`/`--include-wasbcache`/`--summary`); (3) **disk guard** `min_free_gb`; (4) **SessionStart nudge hook** in `.claude/settings.local.json` (local, gitignored). Suite 409?**421 green**, ruff+mypy clean. Policy saved ? [[cache-hygiene-policy]]. ? WSL-scan footgun: the owner's login shell mangles `for e in *` loops (loop var empty) ? use du/find arg-globbing. Also this session (earlier): merged **#107** (API min_shot_count) + **#108** (watermark Windows colon fix); ran GX010125 E2E on GPU ? **GX010125_highlights.mp4** (11 rallies >7 shots) + **GX010125_highlights_min5.mp4** (24 rallies ?5, watermark now works ? validates #108) via a stitch-only reuse script (no re-inference); confirmed **Gemini got 1080p** (4K only at final stitch); delivered an execution flowchart. Manual WSL cleanup freed ~68 GB (kept GX010125's 93 GB frames + proxy per owner choice). ? LOCAL uncommitted `config.json`: default_segmenter=wasb_hybrid / min_shot_count=8 / wasb.timeout_sec=7200 from the run. See [[gotcha-long-runs-gpu-wsl]].

Checkpoint: 2026-06-13 (SYNC + HANDOFF REFRESH ? read-only reconciliation by a ramp-up session; no new code from me this pass). `git pull --ff-only` = already up to date at **master `6de70aa` (#108)**. Reconciled remote vs my last session (#100): seven PRs landed from OTHER slates (NOT my work ? facts from git/gh): **#101** configurable branding watermark · **#102** thread config stitching paddings into live VideoCompiler sites · **#103** bundle Apache-2.0 default watermark font + fallback log · **#104** C13 field kit (flyer/playbook/answer-sheet/signals) · **#105** i18n design plan (Kannada-first) · **#107** enforce `stitching.min_shot_count` on /api/compile · **#108** escape Windows drive-letter colon in ffmpeg filtergraph (fixes #103 watermark on Windows). **OPEN: #106** docs/video-locality-model (awaiting owner). ? **UNCOMMITTED WIP IN TREE (sibling/owner ? DO NOT CLOBBER):** a WSL **cache-hygiene** slice in progress, built on my `WslCommandMixin` ? `base.py` (`_wsl_free_gb`/`_wsl_rm_rf`), `wasb_runner.py`+`tracknet_runner.py` (`keep_frames`/`min_free_gb` config + pre-run disk guard + post-success cache delete), `models.py` (new fields), `config.json` (the GPU-run knobs: default_segmenter=wasb_hybrid, min_shot_count=8, wasb.timeout_sec=7200), untracked `arial.ttf` (watermark font) + `docs/STORAGE_SHARING_MODEL.md`. Not yet PR'd by whoever owns it. [[session-handoff]] rewritten to this state.

Checkpoint: 2026-06-13 (FIRST FULL E2E GPU RUN on a real 4K GoPro match + 2 merged fixes). Owner asked to run the indexer E2E on the GPU over `F:\GoPro Hero Black 12\KhelSutra\Badminton 12 June 2026\GX010125.MP4` (12.8-min 4K60, 11.5GB), highlights for rallies with >7 shots, config-driven.

- **DELIVERED:** `output\GX010125_highlights.mp4` ? **11 rallies (shot_count 8?16) of 52 detected**;
  true 4K 3840×2160 h264 59.94fps, 2:23, 1.15GB; stitched from the 4K ORIGINAL (proxy only for indexing).
  wasb_hybrid: 64.8% shuttle visibility, 18 candidate windows ? 52 Gemini-confirmed rallies. min_shot_count=8
  correctly excluded a 7-shot rally (">7" semantics). Gemini ran clean (no 503s); ~47-min wall clock.
- **2 PRs merged on green CI** (owner's [[working-agreement-merging]] standing authorization):
  #107 `/api/compile` now enforces `stitching.min_shot_count` (parity w/ run_process_and_stitch);
  #108 stitcher `_ff_quote` escapes the Windows drive-letter colon ? fixes the on-by-default
  bundled-font watermark (PR #103) silently failing on Windows (`fontfile='C:/...'` broke ffmpeg
  drawtext parsing; also fixed absolute logo `movie=` paths). Both with regression tests; suite 409 green.
- **The hard-won run recipe is in [[gotcha-long-runs-gpu-wsl]]:** Modern Standby killed the first 2
  overnight attempts (keep-awake fix); cv2 PNG extraction (~38min) is the real bottleneck not the GPU
  (ffmpeg pre-extract, count-exact for CFR ? skips it); 1080p-proxy-index / 4K-stitch; per-phase
  observability (cache manifest for WASB, handoff dir for Gemini). Reusable scripts in `scratch/`
  (run_gx010125.py, keep_awake.ps1, probe_*.py).
- ? LOCAL UNCOMMITTED `config.json`: default_segmenter=wasb_hybrid, min_shot_count=8, wasb.timeout_sec=7200
  (set for this run; revert to gemini/1/1800 if not wanted). Watermark fix NOT applied to the delivered
  reel (it ran before #108); a re-stitch would hit the DB segment cache + now-working watermark (~5min).

Checkpoint: 2026-06-12 (latest ? PAID COMPONENT REMOVED + VIDEO-LOCALITY MODEL; **PR #104 updated `9c84fea` +
PR #106 OPEN `85abe26`, both CI green, both AWAITING OWNER**). Owner spec: (1) remove ALL associate-pay from the
field kit ("something else in mind") ? flyer/playbook now "Engagement terms: discussed individually"; honesty
mechanics reworded ("judged on honest capture, not positive answers"); KEPT: WTP research questions (study subject)
+ the ACADEMY's Phase-2 session fee (different party ? flagged, owner may strip too). (2) Storage/upload questions
answered from code: uploads ARE stored in full (`security.upload_dir`, NO retention policy ? BACKLOG); 10GB CANNOT
upload (max_upload_mb=4096 rejects at init; ?90MB sequential parts; hours on Indian uplinks). (3) NEW
`docs/VIDEO_LOCALITY_MODEL.md` (#106): central insight = timestamps are the product, video is dead weight (only the
compiler touches original bytes, cuttable where the original lives); locality ladder L0?L4 (L2 proxy-upload reuses
the collector ADR-002 frame-identical contract; L1-direct = client?Gemini, zero bytes through us; L0 = owned-model
north star; pilot = L4 sneakernet); **frugality frame (owner): bills affordable iff GPU doesn't burn ? L2/L1 serving
is GPU-less by construction, Gemini ¢/hr, GPU stays on the owned 2070S, per-video cost ceiling required**;
**academy-visit dividend: Phase-2 consented recordings ARE the multi-court target corpus, show-back = live L2 dress
rehearsal, demo reactions = build backlog**; 10 open questions OQ1?OQ10 (retention, MD5 identity, Gemini consent,
key custody, packaging, review source, reel delivery, uplink data, volume, L3 survival); rec lean = L2 default
(NOT a greenlight). NB: #106's base commit was pushed by owner/sibling mid-session

(shared checkout). ?? owner:
377 review/merge #104 + #106; decide academy-fee keep/strip; the "something else" comp structure is his to define.
378
379 **Checkpoint:** 2026-06-12 (latest ? C13 KIT REFRAMED PRODUCT-FEEDBACK-FIRST per owner spec + **RECRUITING FLYER
380 added; PR #104 UPDATED `985a329`, CI 4/4 green, AWAITING OWNER REVIEW** ? do NOT merge without him). Owner spec:
381 hire someone to visit academies, talk to coaches AND students (how does it help + improvement feedback); stretch =
382 record ? pipeline ? show the academy ITS OWN sample result; UI/hosting "ready by then". Kit now = 4 docs by audience:
383 **`PRODUCT_FIELD_ASSOCIATE_FLYER.md`** (NEW ? candidate-facing recruiting flyer, owner fills ?/phone + recruits) ·
384 **`FIELD_VISIT_PLAYBOOK.md`** (RENAMED from FIELD_RESEARCH_ASSOCIATE_FLYER ? day-one playbook: demo-after-Q6 rule,
385 adult-18+-students-only w/ coach-points-out gating, Phase-2 show-back + scripted "can we keep using it?" no-promise
386 reply) · answer sheet (+demo-reaction/student blocks) · PMF_FIELD_SIGNALS (verdict machinery UNCHANGED; demo/student
387 data = product feedback only). 2nd adversarial critique (`w5aexakz2`, 30 agents): 12 confirmed+fixed (key honesty
388 fixes: "web app IS opening"?"we're building?will be able to try"; "match report"?"simple summary"; "finds EVERY
389 rally" dropped; minors loophole in "senior/adult students" closed), 15 refuted. ?? owner: review #104 (thresholds
390 §3 + flyer copy) ? merge ? fill ?/phone ? trim 60s demo reel ? recruit.
391 Owner ratified in-chat: **PMF validation = physically visiting Bangalore academies and reading signals.** Built the
392 decision layer on top of the existing `docs/FIELD_RESEARCH_ASSOCIATE_FLYER.md` (which IS the "recruit hand-out" the
393 owner remembered; the khelacharya PDFs in OneDrive are the separate internship flyer ? see
394 [[sibling-internship-repo]]): **`docs/PMF_FIELD_SIGNALS.md`** (H1-H7 ? script-question map, G/A/R anchors, H5
395 counting rule = pre-ladder-or-budget-converted ??300 only ? anchored ?300 NEVER counts, pinned stopping rule n=15
396 target/logistics-only/verdict-once, numeric verdicts PROCEED / PIVOT A record+index / PIVOT B partner-API
397 (PB Vision×PodPlay footnote now in-repo) / PIVOT C parent-payer / FALSIFIED) + **`docs/FIELD_VISIT_ANSWER_SHEET.md`**
398 (printable per-visit form: verbatim answers, required most-negative quote, before/after-ladder tick, OFFICE-USE
399 score grid) + flyer Q5 rewritten open-ended-first (old wording read the ?100/300/500 ladder ALOUD ? center-bias
400 manufactures the exact ?300 PROCEED threshold; caught by the 38-agent adversarial critique `ww0rvaqf3`: 1 high/2 med/
401 2 low confirmed+fixed, 9 refuted) + Stupa added to listen-for + NEXT_STEPS item 5 = kit COMPLETE. ?? owner: review
402 #104 thresholds ? merge ? fill flyer ?/phone placeholders ? recruit the associate ? first 2 visits paired. PDFs of
403 flyer+answer-sheet: generate AFTER placeholders are filled (offer stands).
404
405 **Checkpoint:** 2026-06-12 (latest ? ROORA VENDOR LOOP COMPLETE: **collector #23+#24+#25 MERGED** sequentially on
406 green CI, owner pre-authorized; master `ca61405`, 0 open PRs, only owner's `.gitignore` edit uncommitted).
407 **#23 boundary_grid_snap BLOCKER** in detect_issues (gcd-of-boundary-frame-diffs > ±0.3s bar ? gate BLOCKED; skips
408 CSV-path/no-fps/<3 rallies) ? ? **replayed against the REAL 6-June PB+TT pilot zips: both auto-block with "All 12
409 rally boundaries sit on a 30-frame grid (~0.50s @ 59.94fps)"** (the manual forensics, now automated) + ROORA_BRIEF
410 Phase-B revision (§7 REQUIRED method: CVAT step 1, first/last box on exact contact/dead frame, sparse interior OK;

§4 grid-snap auto-detected warning; §2 skip-rallies-in-progress-at-file-edges; §1 annotation-optimized-copy note). **#24 prep-annotation-batch**: mirrors `<input-dir>/<match-folder>/<video>` ? proxies-only tree w/ same layout (thin orchestrator over the hardened single flow; per-video isolation; resumable "have" skips; live 2-court E2E smoke green). **#25 ROORA_RUNBOOK.md** (docs/annotation/): operator E2E ? send checklist (layout?batch?manifest commit?intake+email w/ the 2 non-negotiables), receive (gate decision table incl. grid-snap, review sheet, import/export on ORIGINALS), troubleshooting table for every guard. Suite **180 green**. ?? Vendor loop is now fully tooled+documented. ?? AGREED NEXT (owner, session wrap 2026-06-12): (1) owner fills the canonical doc's TODO(avidullu) markers + screenshots, labels it **v8**; (2) **joint TEST RUN of the full vendor loop on real footage BEFORE anything goes to Roora** (runbook §1 send-side end-to-end: batch layout ? prep-annotation-batch ? manifest ? mock-receive via validate-delivery on an old delivery); (3) only then send. **v7 ANOMALY RESOLVED (2026-06-12):** the canonical vendor-facing Brief+Guide is the Google Doc at docs.google.com/document/d/1NvAXE0q7t4Z0QddLkEfnta0DU982yDZaqkGNPMe8EY0 (provenance: generated from master `ca61405`; supersedes Drive v3/v5/v6/v7; owner confirmed superset-of-v7) ? see [[roora-guidelines-drive-doc]]; before sending: pick ONE version label + fill its TODO(avidullu) markers/screenshots.

Checkpoint: 2026-06-12 (earlier ? ADR-002 IMPLEMENTED: **collector #22 MERGED** `a068fd1`, owner authorized merge-on-green; #21 ADR + #20 + indexer #102/#103 all merged earlier today, slate was clean). Shipped: `curate prep-annotation-proxy` + `src/core/proxy.py` ? ?1080p/`-g 15`/CRF 19/audio-kept re-encode w/ `-fps_mode passthrough` (NEVER `-r`), post-encode parity verify (exact frame count; fps ±0.1%; duration like-with-like stream/container), delete-on-ANY-failure (incl. Ctrl-C), VFR refusal w/o `--allow-vfr`, provenance ? `data/annotation_proxy_manifest.csv` (source+proxy MD5s; video_id collision guard BEFORE encode; Excel-lock prints row + keeps proxy), same-file guard normcase/realpath/samefile (NTFS case-variant `--out`+`--force` would have let `ffmpeg -y` TRUNCATE the source ? review verifier reproduced it live pre-fix). Suite **169 green** (+32), CI green. Verified live on ffmpeg 8.1.1 (smoke clip 59.94fps: parity 179 frames both files; smoke caught `-vsync` deprecation ? `-fps_mode`). Process: ultracode adversarial review workflow (`wyv8524zf`, 18 agents) confirmed 1 high/2 med/7 low ? all fixed pre-merge. Docs: INGESTION_PIPELINE step 0 + diagram leg + onboarding FAQ + sampling-FAQ split (highlights-vs-golden), ADR-002 provenance bullet as-implemented amendment, PIPELINE_ROLE ADR refs qualified. ?? REMAINING ADR-002 follow-ups (separate items, NOT started): Phase-B brief revision (frame step 1 / exact boundary frames / "we supply proxies") ? owner words vendor comms; grid-snap QA check in validate-delivery. First real use: run prep-annotation-proxy on the next Roora batch before handoff.

Checkpoint: 2026-06-12 (earlier ? ANNOTATION-PROXY DECISION RATIFIED ? **collector PR #21 OPEN**, docs-only **ADR-002** in collector `docs/DECISIONS.md`). Owner agreed in-chat w/ my rec from the assessment checkpoint below: proxies generated in-repo via a future `curate prep-annotation-proxy`. ADR records the proxy contract (SAME fps ? never reduce; no trim; frame-count parity check; 1080p/CRF18-20/-g15/audio kept; intake-manifest row w/ original+proxy MD5s), the why-collector rationale (owns the vendor flow = command #0 of convert-

cvat?validate-delivery?import-local;
verification+provenance is the durable 80%, one-off scripts skip it), and the **Phase-B necessity argument the owner
wanted captured: ~12-15 videos/sport × ~30 min 4K/59.94 = >100k frames/file ? vendor frame-step-1 annotation
infeasible without proxies ? near-precondition, not optimization**. Follow-ups recorded IN the ADR: Phase-B brief
convention line (step=1, first/last box on exact contact/dead frame) + re-add grid-snap QA to validate-delivery.
Collector checkout back on master (owner's stray `.gitignore` edit untouched). ? sibling PR #20 (below) edits
rally_cvat.py docstring rationale 0.5/1.0?0.5/0.5 ? no file overlap w/ #21, both docs-true. ?? owner reviews/merges

6. assistant (2026-06-14T10:31:16.622Z)

This is the owner's real GPU machine, and the task is well-scoped. Let me orient on the tooling and environment before doing anything. Let me read the key files and check what's cached.

7. user (2026-06-14T10:31:17.593Z)

```
"""Run the framework on GX010125_1080p by REUSING its cached WASB trajectory ? offline,
no GPU, no 4K, isolated outputs (output/GX010125_run/), zero pollution of past work.

This is the "persist detection once, re-score forever" path on a REAL, unlabeled match:
parse the cached 1080p trajectory -> candidate rally windows -> CONTROL vs Gate-A
(net-crossings >= 2, the measured precision win) side-by-side (no golden labels exist for
this footage, so we report the divergence, not F1).
"""
import json
import os

from backend.eval.distill_local import build_candidates, FEATURE_NAMES
from backend.eval.experiment import CONTROL, Variant, VideoCandidates, compare_unlabeled, predict

TRAJ = "output/wasb/GX010125_1080p__89605e80ed01_wasb.csv"
FPS, FRAME_W = 59.94, 1920.0
OUT_DIR = "output/GX010125_run"


def _dur(ivs):
    return sum(e - s for s, e in ivs)


def _write_csv(path, ivs):
    with open(path, "w", encoding="utf-8") as f:
        f.write("start_sec,end_sec,duration_sec\n")
        for s, e in ivs:
            f.write(f"{s:.3f},{e:.3f},{e - s:.3f}\n")


def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    intervals, X = build_candidates(TRAJ, FPS, FRAME_W)
    feats = {FEATURE_NAMES[k]: [r[k] for r in X] for k in range(len(FEATURE_NAMES))}
    vc = VideoCandidates(name="GX010125_1080p", intervals=intervals, features=feats, gts=None)

    gateA = Variant(name="gateA", min_crossings=2)
    ctrl_preds = predict(CONTROL, vc)
    gate_preds = predict(gateA, vc)
    sxs = compare_unlabeled(vc, CONTROL, gateA)   # agreed / a_only (dropped by Gate-A) / b_only
```

```
        print("=" * 70)
        print(f"GX010125_1080p ? offline re-score of the CACHED trajectory (no GPU)")
        print("=" * 70)
        print(f"  candidate windows from trajectory : {len(intervals)}")
        print(f"  CONTROL rallies (merge all)       : {len(ctrl_preds)}  ({_dur(ctrl_preds) / 60:.1f} min)")
        print(f"  Gate-A rallies (net_crossings>=2) : {len(gate_preds)}  ({_dur(gate_preds) / 60:.1f} min)")
        print(f"  Gate-A dropped (a_only)           : {len(sxs['a_only'])} windows "
              f"({_dur(sxs['a_only']) / 60:.1f} min of likely held-shuttle/feed FPs)")
        print(f"  both agree (kept)                 : {sxs['num_agreed']}")

        _write_csv(os.path.join(OUT_DIR, "GX010125_control_rallies.csv"), ctrl_preds)
        _write_csv(os.path.join(OUT_DIR, "GX010125_gateA_rallies.csv"), gate_preds)
        summary = {
            "video": "GX010125_1080p", "source": "cached WASB trajectory (offline re-score, no GPU)",
            "fps": FPS, "num_candidates": len(intervals),
            "control": {"num_rallies": len(ctrl_preds), "total_min": round(_dur(ctrl_preds) / 60, 2)},
            "gateA": {"num_rallies": len(gate_preds), "total_min": round(_dur(gate_preds) / 60, 2),
                      "dropped_windows": len(sxs["a_only"])},
        }
        json.dump(summary, open(os.path.join(OUT_DIR, "GX010125_rescore_summary.json"), "w"), indent=2)
        print(f"\n  wrote isolated outputs -> {OUT_DIR}/  (control + gateA rally CSVs + summary)")
        print("  NOTE: no golden labels for this footage -> divergence reported, not F1. A full")
        print("  Gemini-confirmed highlight reel is a separate ~tens-of-min GPU+API run.")


    if __name__ == "__main__":
        main()

```

8. user (2026-06-14T10:31:18.042Z)

```
"""Distill Gemini (teacher) into a LEARNED local rally classifier (student).

Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
using a few resolution/fps-INVARIANT local features, so the model transfers across
videos and runs 100% local at inference (no Gemini, privacy lever intact).

Flow (all local at runtime):
  WASB trajectory -> recall-oriented candidate windows (propose many; classifier
  filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
  active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
  -> rally windows.

Training labels come from Gemini full-context labels via the SAME IoU matching the
evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
other videos' candidates, predict the held-out one, score vs its Gemini labels, and
compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.

Run:
    python -m backend.eval.distill_local --manifest videos.json --model-out output/local_rally_model.json
"""
from __future__ import annotations

```

```
import json
import argparse
from typing import Dict, List, Optional, Tuple


from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.detectors import rally_gate
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.eval.classifier import LogisticRegression
from backend.eval.metrics import score
from backend.eval.gemini_refine import merge_overlaps

Interval = Tuple[float, float]

FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]
# Recall-oriented proposal: deliberately permissive so real rallies are among the
# candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
PROPOSAL = (0.005, 1.0, 0.5)
BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish windowing, no learning


def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                     proposal: Tuple[float, float, float] = PROPOSAL) -> Tuple[List[Interval], List[List[float]]]:
    """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature rows)."""
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = proposal
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
    intervals = [(w["start"], w["end"]) for w in wins]
    gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-crossings
    X: List[List[float]] = []
    for w, g in zip(wins, gated):
        dur = max(1e-6, w["end"] - w["start"])
        win_frames = max(1.0, dur * fps)
        X.append([
            dur,                                              # rally length (sec) ? fps-invariant
            float(w["peak_velocity"]),                        # already normalized by frame_width
            float(w["mean_velocity"]),
            float(w["active_frame_count"]) / win_frames,      # active-shuttle ratio ? fps-invariant
            float(g.crossings),                               # net crossings (count) ? the rally signal
        ])
    return intervals, X


def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) -> List[Interval]:
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = BASELINE_LOCAL
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
    return [(w["start"], w["end"]) for w in wins]


def load_video(spec: Dict, iou: float = 0.5) -> Dict:
```

```
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    intervals, X = build_candidates(spec["trajectory"], fps, fw)
    gts = load_golden_gts(spec["labels"])
    y = label_candidates(intervals, gts, iou)
    return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X": X,
            "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps, fw)}


def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals: List[Interval],
                    threshold: float = 0.5) -> List[Interval]:
    if not intervals:
        return []
    proba = model.predict_proba(X)
    pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
    return merge_overlaps(pos, gap_tol=1.0)


def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
        model_out: Optional[str] = None) -> dict:
    videos = [load_video(s, iou) for s in specs]
    print(f"=== Distilled local rally classifier vs Gemini silver-GT "
          f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
    for v in videos:
        ceil = score(v["intervals"], v["gts"], iou).recall
        print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
              f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

    result: dict = {"videos": [v["name"] for v in videos]}
    if len(videos) >= 2:
        print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-out) ---")
        clf, base = [], []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]
            X = [row for v in others for row in v["X"]]
            y = [t for v in others for t in v["y"]]
            model = LogisticRegression().fit(X, y, FEATURE_NAMES)
            preds = predict_rallies(model, held["X"], held["intervals"], threshold)
            sc = score(preds, held["gts"], iou)
            bsc = score(held["baseline"], held["gts"], iou)
            clf.append(sc)
            base.append(bsc)
            print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f} R={sc.recall:.3f} F1={sc.f1:.3f}"
                  f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f} F1={bsc.f1:.3f}")
        def mean(xs, a):
            return sum(getattr(s, a) for s in xs) / len(xs)
        print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f} R={mean(clf,'recall'):.3f} "
              f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
              f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
        result["mean_learned_f1"] = mean(clf, "f1")
        result["mean_baseline_f1"] = mean(base, "f1")

    # Final model on ALL videos (for production / running on a fresh held-out video).
    Xall = [row for v in videos for row in v["X"]]
    yall = [t for v in videos for t in v["y"]]
    final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
    assert final.w is not None  # set by fit()
    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
    print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
```

```
    if model_out:
        final.save(model_out)
        print(f"final model (all videos) -> {model_out}")
        result["model_out"] = model_out
    return result


def main() -> None:
    ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally classifier.")
    ap.add_argument("--manifest", required=True,
                    help="JSON: [{name, trajectory, fps, frame_width, labels}, ...] (labels = Gemini CSV)")
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability cutoff")
    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
    with open(args.manifest, encoding="utf-8") as f:
        specs = json.load(f)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)


if __name__ == "__main__":
    main()
```

9. user (2026-06-14T10:31:19.461Z)

```
=== output/wasb CSVs ===
total 1804
drwxr-xr-x 1 avidu 197609       0 Jun 13 14:43 .
drwxr-xr-x 1 avidu 197609       0 Jun 14 11:51 ..
-rw-r--r-- 1 avidu 197609 1635249 Jun 13 14:43 GX010125_1080p__89605e80ed01_wasb.csv
drwxr-xr-x 1 avidu 197609       0 Jun 13 14:42 GX010125_1080p__89605e80ed01_wasb_wasbcache
-rw-r--r-- 1 avidu 197609  172447 Jun 11 18:28 mahadevpura_smoke_180s__05e0e577029d_wasb.csv
drwxr-xr-x 1 avidu 197609       0 Jun 11 18:28
mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache
=== output dirs ===
GX010125_1080p.mp4
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
adarsh_min5s_segments.csv
adarsh_tuned_segments.csv
badminton_indexer.db
e2e_reel.mp4
e2e_server_log.txt
fusion_golden.log
gbaaddy_golden_features.json
gbaaddy_vs_human.csv
gx010125_min5_stitch.log
gx010125_run.log
gx010125_transcode.log
```

human_lovo_manifest.json
kushagra_1280.mp4
kushagra_1280.report.json
kushagra_1280.report.md
kushagra_1280.summary.md
kushagra_singles_golden_features.json
kushagra_vs_human.csv
largetest_doubles_golden_features.json
largetestvideo_vs_human.csv
local_calib_manifest.json
local_rally_model.json
mahadevpura_gemini_refined.csv
mahadevpura_singles_golden_features.json
mahadevpura_smoke_180s.mp4
mahadevpura_smoke_180s.report.json
mahadevpura_smoke_180s.report.md
mahadevpura_smoke_180s.summary.md
mahadevpura_vs_human.csv
phase3_probe.json
phase3_yolo.db
ramp_probe.mp4
smoke4_log.txt
sports_indexer.db
test_indexer.db

10. assistant (2026-06-14T10:31:38.729Z)

I have cached WASB CSVs for GX010125 (unlabeled) and a mahadevpura smoke clip. The golden corpus (with labels) is referenced by the manifest. Let me read the manifest, the FusionSegmenter served path, the experiment harness, and the existing n6 LOVO repro ? plus check the environment.

11. user (2026-06-14T10:31:40.403Z)

=== human_lovo_manifest.json ===
[
  {
    "name": "adarsh_avi_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
    "fps": 59.94, "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and Avi.rallies.csv"
  },
  {
    "name": "kushagra_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/kushagra_traj.csv",
    "fps": 59.94, "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/KushagraYashAviFirstVideo.rallies.csv"
  },
  {
    "name": "largetest_doubles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/largetestvideo_traj.csv",
    "fps": 29.97, "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/LargeTestVideo.rallies.csv"
  },
  {
    "name": "gbaaddy",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/gbaaddy_traj.csv",
    "fps": 59.94, "frame_width": 1920,

```
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv"
  },
  {
    "name": "testlarge_short",
    "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv",
    "fps": 29.97, "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv"
  },
  {
    "name": "mahadevpura_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
    "fps": 29.97, "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv"
  }
]
```


=== config.json (indexing section) ===


```
{
  "default_segmenter": "fusion_hybrid",
  "use_gpu": true,
  "motion_threshold": 0.08,
  "min_segment_duration": 3.0,
  "dead_time_merge_gap": 2.0,
  "chunk_duration": 90,
  "chunk_overlap": 10,
  "detector_model": "fasterrcnn_resnet50_fpn",
  "detector_score_threshold": 0.5,
  "yolo_velocity_threshold": 0.15,
  "yolo_frame_skip": 3,
  "yolo_tracker": "bytetrack.yaml",
  "yolo_prefetch_workers": 2,
  "min_action_window_duration": 2.0,
  "log_yolo_candidates": true,
  "store_yolo_candidates": true,
  "ai_handoff_padding": 2.0,
  "ai_max_workers": 1,
  "tracknet": {
    "wsl_distro": "Ubuntu",
    "repo_dir": "~/models/TrackNetV4",
    "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
    "conda_env": "TrackNetV4",
    "weights_path": "",
    "queue_length": 5,
    "stage_in_wsl": true,
    "wsl_stage_dir": "~/clips",
    "timeout_sec": 1800,
    "velocity_threshold": 0.02
  },
  "wasb": {
    "wsl_distro": "Ubuntu",
    "repo_dir": "~/models/WASB-SBDT",
    "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
    "conda_env": "wasb",
    "weights_path": "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
    "sport": "badminton",
    "wsl_stage_dir": "~/clips",
    "timeout_sec": 1800,
    "velocity_threshold": 0.02
  },
  "fusion": {
```

```
    "substrate": "wasb",
    "min_crossings": 2,
    "min_players": 0,
    "cue_flags": {}
  }
}
```


12. user (2026-06-14T10:31:41.091Z)

```
"""Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
shuttle-seeded window via two overridable seams the engine exposes:

- ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
  GPU-free, computed from the trajectory we already have); and, only when the person cue
  is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
  (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
  fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads the
  torchvision/supervision model ? so the default (person-OFF) path never loads a detector
  model nor touches the GPU. (torch/cv2 themselves are imported by the registry regardless,
  via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
- ``_select_for_handoff`` ? optional served Gate A/B: when ``indexing.fusion.min_crossings``
  (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows are
  dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
  **Persisted candidates remain the full superset** regardless, so the offline experiment
  harness can re-score any variant.

Defaults reproduce the substrate exactly: net_crossings/person features are persisted but
nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config seeds
and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the default
segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift through the
experiment harness (EXPERIMENT_HARNESS.md), never silently.
"""
from typing import Any, Dict, List, Optional, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors import fusion_features as ff


class FusionSegmenter(TrajectoryHybridSegmenter):
    """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

    family = "fusion"
    display_label = "Fusion"

    def __init__(self, db: Any, config: Any):
```

```
        super().__init__(db, config)
        # Built lazily only if the person cue is enabled (no detector model loaded otherwise).
        self._person: Optional[Any] = None
        # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-estimate it
        # over the whole trajectory once per candidate window (it depends only on `points`).
        self._axis_cache_key: Optional[int] = None
        self._axis_cache: Optional[tuple] = None

    @property
    def name(self) -> str:
        return "fusion_hybrid"

    @property
    def description(self) -> str:
        return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
                "AI provider sets semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
        if substrate == "tracknet":
            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
            return TrackNetRunner(cfg), {
                "weights_path": cfg.weights_path,
                "queue_length": cfg.queue_length,
                "fusion_substrate": "tracknet",
            }
        wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
        # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always persisted
        # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-estimated
        # over the whole trajectory by rally_gate (camera-agnostic); reused across windows.
        gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                        min_crossings=0, estimate=self._axis_estimate(points))
        stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

        fcfg = idx_cfg.get("fusion", {})
        if fcfg.get("cue_flags", {}).get("person", False):
            stats.update(self._person_features(cw, points, fps, frame_width, video_path, fcfg))
        return stats

    def _axis_estimate(self, points: List[Any]) -> tuple:
        """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so it's
        computed once per video, not once per candidate window (it depends only on points)."""
        key = id(points)
```

```
        if self._axis_cache_key != key or self._axis_cache is None:
            self._axis_cache_key = key
            self._axis_cache = rally_gate.estimate_play_axis(points)
        est = self._axis_cache
        assert est is not None
        return est

    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                         frame_width: float, video_path: str,
                         fcfg: Dict[str, Any]) -> Dict[str, float]:
        """GPU path (person cue ON): run the person detector over this window's ORIGINAL-video
        frame range and compute the scale/translation-invariant person features. Lazily builds
        ONE PersonDetector. ? real-video verification is owner-side; this wiring is mock-tested."""
        # Lazy import so the default (person-OFF) path never pulls torch/cv2 through this module.
        from backend.pipeline.detectors.person_detector import PersonDetector
        if self._person is None:
            self._person = PersonDetector(self.config)

        frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
        # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
        # window-edge off-by-one that drops the boundary frame from the shuttle alignment.
        start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
        det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
        fw = det["frame_width"] or frame_width
        fh = det["frame_height"]
        # If we can't trust the frame dims, every normalised feature would silently collapse
        # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked one.
        if fw <= 0 or fh <= 0:
            return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

        poly = fcfg.get("court_polygon")
        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                          court_polygon=polygon, net=net)

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Served Gate A/B over the (already-enriched) per-window stats. Default thresholds
        (0) keep every window ? identical to the substrate. Persisted candidates are
        untouched (full superset)."""
        fcfg = idx_cfg.get("fusion", {})
        min_crossings = int(fcfg.get("min_crossings", 0))
        min_players = int(fcfg.get("min_players", 0))
        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))

        keep: List[int] = []
        for i, st in enumerate(window_stats):
            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
                continue
            if person_on and min_players > 0 and st.get("players_in_court", 0.0) < min_players:
                continue
            keep.append(i)
        return keep
```

```


segmenter_registry.register(FusionSegmenter)

```

13. user (2026-06-14T10:31:41.485Z)

```
"""First full A/B experiment on the n=6 golden corpus ? runs entirely offline
(persisted WASB trajectories, no GPU). One consistent candidate set per video
(recall-oriented proposal windowing, the persisted pipeline's own), re-scored three
ways, then graded with the C1 honest aggregation + C4 segmentation metrics + a C3
out-of-fold learned mAP. Prints a report and writes output/ab_experiment_n6.json.

Honest: prints whatever the real golden data produces. No claims before measurement.
"""
import json

from backend.eval.distill_local import FEATURE_NAMES, build_candidates
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.eval.classifier import LogisticRegression
from backend.eval.experiment import CONTROL, Variant, VideoCandidates, evaluate_variant
from backend.eval.metrics import score
from backend.eval import segmentation_metrics as sm
from backend.eval import eval_stats as es

IOU = 0.5
MANIFEST = "output/human_lovo_manifest.json"
OUT = "output/ab_experiment_n6.json"


def load_corpus():
    specs = json.load(open(MANIFEST))
    videos, pooled = [], {"X": [], "y": [], "group": [], "interval": []}
    for s in specs:
        fps, fw = float(s["fps"]), float(s["frame_width"])
        intervals, X = build_candidates(s["trajectory"], fps, fw)
        gts = load_golden_gts(s["labels"])
        y = label_candidates(intervals, gts, IOU)
        feats = {FEATURE_NAMES[k]: [row[k] for row in X] for k in range(len(FEATURE_NAMES))}
        videos.append(VideoCandidates(name=s["name"], intervals=intervals, features=feats, gts=gts))
        for iv, row, yi in zip(intervals, X, y):
            pooled["X"].append(row); pooled["y"].append(yi)
            pooled["group"].append(s["name"]); pooled["interval"].append(iv)
    return videos, pooled


def c4_aggregate(ev, videos):
    """Mean F1@k + mean segment-count ratio across videos for one variant's predictions."""
    by_name = {v.name: v for v in videos}
    f1k = {0.1: [], 0.25: [], 0.5: []}
    ratios = []
    for name, preds in ev["predictions"].items():
        gts = by_name[name].gts
        got = sm.f1_at_overlaps(preds, gts)
        for k in f1k:
            f1k[k].append(got[k])
        ratios.append(sm.segment_count_ratio(preds, gts))
    mean = lambda xs: sum(xs) / len(xs) if xs else 0.0
    return {f"f1@{int(k*100)}": round(mean(v), 3) for k, v in f1k.items()} | \
           {"seg_count_ratio": round(mean(ratios), 2)}

```

```python

def oof_learned_map(pooled, videos):
    """C3 + C4: out-of-fold learned candidate proba ? mAP@tIoU per video, averaged."""
    def fit_predict(train_idx, test_idx):
        m = LogisticRegression().fit([pooled["X"][i] for i in train_idx],
                                     [pooled["y"][i] for i in train_idx], FEATURE_NAMES)
        return list(m.predict_proba([pooled["X"][i] for i in test_idx]))
    oof = es.out_of_fold_predictions(pooled["group"], fit_predict)
    by_name = {v.name: v for v in videos}
    maps = []
    for name in by_name:
        scored = [(pooled["interval"][i], float(oof[i]))
                  for i in range(len(oof)) if pooled["group"][i] == name and oof[i] is not None]
        maps.append(sm.mean_average_precision(scored, by_name[name].gts))
    return sum(maps) / len(maps) if maps else 0.0


def c1_block(a_f1, b_f1):
    d = es.paired_delta_ci(a_f1, b_f1, seed=0)
    return {
        "mean_a": round(sum(a_f1) / len(a_f1), 3),
        "mean_b": round(sum(b_f1) / len(b_f1), 3),
        "mean_delta": round(d["mean_delta"], 3),
        "ci": [round(d["ci_low"], 3), round(d["ci_high"], 3)],
        "ci_excludes_zero": d["ci_excludes_zero"],
        "sign": {k: int(v) if k != "p_value" else round(v, 3) for k, v in d["sign_test"].items()},
    }


def main():
    videos, pooled = load_corpus()
    print("=" * 80)
    print(f"FIRST A/B EXPERIMENT ? n={len(videos)} golden corpus (offline, persisted trajectories)")
    print("=" * 80)
    for v in videos:
        ceil = score(v.intervals, v.gts, IOU).recall
        print(f"  {v.name:<22} {len(v.intervals):>4} candidates  {len(v.gts):>3} rallies "
              f"proposal-recall-ceiling={ceil:.3f}")

    control = evaluate_variant(videos, CONTROL, IOU)
    gateA = evaluate_variant(videos, Variant(name="gateA", min_crossings=2), IOU)
    learned = evaluate_variant(videos, Variant(name="learned", feature_names=FEATURE_NAMES), IOU, honest=True)

    order = [p["video"] for p in control["per_video"]]
    f1 = {"CONTROL": {p["video"]: p["f1"] for p in control["per_video"]},
          "gateA": {p["video"]: p["f1"] for p in gateA["per_video"]},
          "learned": {p["video"]: p["f1"] for p in learned["per_video"]}}

    def col(name):
        return [f1[name][v] for v in order]

    print("\n--- per-video held-out F1 ---")
    print(f"  {'video':<22}{'CONTROL':>9}{'GateA':>9}{'learned':>9}")
    for v in order:
        print(f" {v:<22}{f1['CONTROL'][v]:>9.3f}{f1['gateA'][v]:>9.3f}{f1['learned'][v]:>9.3f}")

    ab_gate = c1_block(col("CONTROL"), col("gateA"))
    ab_learned = c1_block(col("CONTROL"), col("learned"))
    print("\n--- A/B 1: CONTROL vs Gate-A (static) ? C1 honest aggregation ---")
```

```
    print(f"  meanF1 {ab_gate['mean_a']} -> {ab_gate['mean_b']}
?F1={ab_gate['mean_delta']:+} "
          f"CI{ab_gate['ci']} excl0={ab_gate['ci_excludes_zero']}
sign={ab_gate['sign']}")
    print("--- A/B 2: CONTROL vs learned fusion (LOVO) ? C1 honest aggregation ---")
    print(f"  meanF1 {ab_learned['mean_a']} -> {ab_learned['mean_b']}
?F1={ab_learned['mean_delta']:+} "
          f"CI{ab_learned['ci']} excl0={ab_learned['ci_excludes_zero']}
sign={ab_learned['sign']}")

    c4 = {name: c4_aggregate(ev, videos)
          for name, ev in (("CONTROL", control), ("gateA", gateA), ("learned",
learned))}
    print("\n--- C4 over-segmentation-aware metrics (mean across videos) ---")
    for name in ("CONTROL", "gateA", "learned"):
        print(f"  {name:<9} {c4[name]}")
    oof_map = oof_learned_map(pooled, videos)
    print(f"  learned out-of-fold (C3) mAP@tIoU[.3-.7] = {oof_map:.3f}")

    result = {
        "corpus": order, "n": len(videos), "iou": IOU,
        "aggregate_f1": {k: round(sum(col(k)) / len(order), 3) for k in ("CONTROL",
"gateA", "learned")},
        "per_video_f1": f1,
        "ab_control_vs_gateA": ab_gate,
        "ab_control_vs_learned": ab_learned,
        "c4_metrics": c4,
        "learned_oof_map": round(oof_map, 3),
    }
    json.dump(result, open(OUT, "w"), indent=2)
    print(f"\nwrote {OUT}")


if __name__ == "__main__":
    main()

```

14. assistant (2026-06-14T10:32:19.249Z)

Now I understand the architecture. The key realization: the offline harness (`build_candidates` + `experiment.predict`) reimplements gating but does NOT exercise the real `FusionSegmenter._enrich_candidate_stats`/`_select_for_handoff` served code. To close the served-side gap properly, I should drive the **actual** `FusionSegmenter` methods over the cached trajectory. Let me read the base engine to understand the production windowing path and whether I can drive it offline.

15. user (2026-06-14T10:32:19.919Z)

```
"""Shared engine for trajectory-substrate hybrid segmenters.

tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
now the single home for the whole pipeline shape:

    healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
    -> P3 AI-provider handoff over padded clips -> P4 DB population/linking

A concrete subclass supplies only its identity and runner wiring:
- ``family``        ? config section under ``indexing.*``, output subdir, and
                      the ``<family>-hybrid-<model>`` detector tag in metadata.
- ``display_label`` ? human label for log lines ("WASB", "TrackNet").
- ``name`` / ``description`` properties (the registry contract).
- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                      detection_params extras such as weights_path).

```

```
This base is deliberately NOT registered; only concrete subclasses register.
"""
import os
import time
import logging
import concurrent.futures
from typing import Any, Dict, List, Optional, Tuple

import cv2

from backend.pipeline.segmenters.base import BasePlaySegmenter
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


class TrajectoryHybridSegmenter(BasePlaySegmenter):
    """Trajectory-detector hybrid: precise candidate rallies + AI semantic boundaries."""

    #: config section under `indexing` (velocity_threshold lives there), the
    #: output subdir for trajectories, and the metadata detector-tag prefix.
    family: str = ""
    #: human-readable label used in log lines.
    display_label: str = ""

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, detector-specific detection_params extras)."""
        raise NotImplementedError

    @staticmethod
    def _probe_fps_width(video_path: str) -> tuple:
        """Return (fps, frame_width) for a video, with sensible fallbacks."""
        cap = cv2.VideoCapture(video_path)
        fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
        width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
        cap.release()
        return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

    @staticmethod
    def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
        """Provider-reported exchange count, else a duration-based estimate.

        One canonical implementation ? the twins had drifted (wasb relied on
        ``int(None)`` raising into the fallback; same result, by accident).
        """
        shot_count = segment.get("shuttle_exchange_count")
        if shot_count is None:
            return max(3, int(duration / 0.8))
        try:
            return int(shot_count)
        except (ValueError, TypeError):
            return max(3, int(duration / 0.8))

```

```
    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
        return {
            "peak_velocity": cw["peak_velocity"],
            "mean_velocity": cw["mean_velocity"],
            "active_frame_count": cw["active_frame_count"],
            "track_id_count": cw["track_id_count"],
        }

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Indices of the candidate windows that proceed to the AI handoff (and thus may
        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
        Persisted candidates are NOT affected ? they remain the full superset for offline
        re-scoring."""
        return list(range(len(windows)))

    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                      player_pool: Optional[List[str]] = None,
                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
        # override-ignore: the ABC declares the Result contract (Success|Failure)
        # but every live segmenter still returns a bare list ? the documented
        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
        label = self.display_label
        # --- Sport profile + AI provider wiring (identical across segmenters) ---
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
            return []

        idx_cfg = self.config.get("indexing", {})
        provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
        model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))

        api_key_env_var = f"{provider_name.upper()}_API_KEY"
        api_key = os.environ.get(api_key_env_var)
        if not api_key:
            logger.error(
                f"{api_key_env_var} environment variable is not set! "
                f"To run the {provider_name} handoff, set your API key. "
                f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
            )
            return []
        try:
            provider = provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
        except KeyError as e:
            logger.error(str(e))
```

```
            return []

        video_duration = get_video_duration(video_path)
        if video_duration <= 0:
            logger.error("Invalid video duration.")
            return []
        fps, frame_width = self._probe_fps_width(video_path)

        # --- Runner config + windowing thresholds ---
        runner, runner_params = self._build_runner(idx_cfg)

        velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
        merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
        min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
        handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
        ai_max_workers = idx_cfg.get("ai_max_workers", 5)
        log_candidates = idx_cfg.get("log_yolo_candidates", True)
        store_candidates = idx_cfg.get("store_yolo_candidates", True)

        detection_params = {
            "detector": self.name,
            "velocity_thresh": velocity_thresh,
            "merge_gap": merge_gap,
            "min_action_window_duration": min_window_duration,
            **runner_params,
        }

        # --- Phase 1: env healthcheck (fail fast with a clear message) ---
        ok, msg = runner.healthcheck()
        if not ok:
            logger.error("%s WSL environment not ready: %s", label, msg)
            return []
        logger.info("%s WSL env OK: %s", label, msg)

        # --- Phase 2: trajectory -> candidate rally windows ---
        # Trajectory detectors process the whole video in one pass (they carry
        # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
        t_start = time.time()
        out_dir = os.path.join("output", self.family)
        csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
        if not csv_path:
            logger.error("%s produced no trajectory; aborting.", label)
            return []

        points = runner.parse_trajectory_csv(csv_path)
        logger.info("Parsed %d trajectory points (%d visible).",
                    len(points), sum(1 for p in points if p.visible))

        all_candidate_windows = runner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width, chunk_start=0.0,
            velocity_thresh=velocity_thresh, merge_gap=merge_gap,
            min_window_duration=min_window_duration, chunk_index=0,
        )
        logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                    label, time.time() - t_start, len(all_candidate_windows))

        if log_candidates:
            for cw in all_candidate_windows:
                logger.info(f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                            f"({cw['end'] - cw['start']:.1f}s) | frames={cw['active_frame_count']} "
                            f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}")

```

```
        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
        # persistence and the handoff gate below.
        window_stats = [
            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
            for cw in all_candidate_windows
        ]

        # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
        candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
        if store_candidates:
            for i, cw in enumerate(all_candidate_windows):
                candidate_ids[i] = self.db.add_candidate_segment(
                    video_id, detector=self.name,
                    start_time=cw["start"], end_time=cw["end"],
                    chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
                    stats=window_stats[i], detection_params=detection_params,
                )

        if not all_candidate_windows:
            return []

        # --- Phase 3: AI provider handoff over padded candidate clips ---
        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
        # candidates above stay the full superset ? only the served play_segments are gated.
        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats, idx_cfg)
        handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
        os.makedirs(handoff_dir, exist_ok=True)
        logger.info("Phase 3: %s validation on %d candidate windows...",
                    provider_name, len(handoff_indices))

        def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
            w_start = max(0, window["start"] - handoff_padding)
            w_end = min(video_duration, window["end"] + handoff_padding)
            w_dur = w_end - w_start
            w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
            if not extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
                return []
            prompt = sport_profile.get_prompt(chunk_duration=w_dur)
            segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                 label=f"Handoff {wi+1}")
            valid = []
            for seg in segments:
                try:
                    seg["start_time"] = float(seg["start_time"]) + w_start
                    seg["end_time"] = float(seg["end_time"]) + w_start
                    seg["_candidate_id"] = candidate_ids[wi]
                    valid.append(seg)
                except (ValueError, TypeError, KeyError) as e:
                    logger.warning("Dropping segment with unparseable timestamps: %s ? %s", seg, e)
            try:
                os.remove(w_path)
            except OSError:
                pass
            return valid

```

```
        ai_segments: List[dict] = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
                       for wi in handoff_indices]
            for future in futures:
                ai_segments.extend(future.result())
        try:
            os.rmdir(handoff_dir)
        except OSError:
            pass
        logger.info("Phase 3 completed. AI confirmed %d semantic play segments.", len(ai_segments))

        # --- Phase 4: DB population + candidate linking ---
        segments_data = []
        for segment in ai_segments:
            try:
                start = float(segment["start_time"])
                end = float(segment["end_time"])
                dur = end - start
                metadata = {
                    "shot_count": self._shot_count_from(segment, dur),
                    "shot_count_confidence": segment.get("shot_count_confidence", "Unknown"),
                    "detector": f"{self.family}-hybrid-{model_name}",
                }
                segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
                if segment_id:
                    candidate_id = segment.get("_candidate_id")
                    if candidate_id is not None:
                        self.db.link_candidate_to_segment(candidate_id, segment_id)
                    segments_data.append({
                        "id": segment_id, "start_time": start, "end_time": end,
                        "duration": dur, "metadata": metadata,
                    })
            except Exception as e:
                logger.warning("Skipping invalid segment struct: %s", e)

        return segments_data

```

16. assistant (2026-06-14T10:33:11.577Z)

Now I fully understand the served path. Critical detail: the **served** windowing uses production params (`velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0` from config), which differ from the offline harness's recall-oriented proposal `(0.005, 1.0, 0.5)`. So the faithful served-side test must window with production params and then call the **real** `FusionSegmenter._enrich_candidate_stats`/`_select_for_handoff`. Let me check how the app loads config and instantiates segmenters so I reproduce production exactly, and confirm the environment (numpy/cv2/rally_gate).

17. user (2026-06-14T10:33:12.994Z)

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Type, Optional
from backend.infrastructure.database import Database
from backend.results import Result  # new standardized contract (Result[Value] == Success[Value] | Failure)

class BasePlaySegmenter(ABC):
    def __init__(self, db: Database, config: Dict[str, Any]):
        self.db = db
```

```
        self.config = config

    @property
    @abstractmethod
    def name(self) -> str:
        """Unique identifier for this segmenter."""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """Human-readable explanation of this segmenter."""
        pass

    @abstractmethod
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments, populates SQLite database,
        and returns a Result.

        Use isinstance(result, Failure) (or result.is_success()) to distinguish
        "no segments found" from "something went wrong" (missing key, healthcheck
        failure, etc.). This is the start of the standardized error/result contract.
        """
        pass

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}: {e}") from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}
        for name, cls in self._segmenters.items():
            try:
                temp_inst = cls(None, {})
                desc = temp_inst.description
            except Exception:
                desc = "No description available."
            res[name] = desc
        return res

segmenter_registry = SegmenterRegistry()

```

18. user (2026-06-14T10:33:15.064Z)

=== how config is loaded + segmenter instantiated in run_process_and_stitch ===

19. user (2026-06-14T10:33:15.174Z)

```python
"""Rally-start gate: suppress between-point false fires using shuttle net-crossings.

WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
the opponent for service after losing a point, that single trajectory looks like a
rally. The discriminator is structural, not duration-based: a real rally has the
shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
single service feed crosses ~once.

This module is camera-agnostic and needs NO new model ? it works on the trajectory
we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
axis with the larger shuttle spread) and the net position (median shuttle coord on
that axis ? the shuttle clusters at the two ends and crosses the net region often,
so the median sits near the net), then counts sign changes of (coord - net) across
the visible points inside a candidate window, with a deadband to ignore jitter.

Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
windows from trajectory_to_action_windows, or to fold into it later.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import List, Optional, Sequence, Tuple

Interval = Tuple[float, float]


@dataclass
class GatedWindow:
    start: float
    end: float
    crossings: int
    visible_points: int
    kept: bool


def _spread(vals: Sequence[float]) -> float:
    """Robust spread = p95 - p5 (ignores a few outlier detections)."""
    if len(vals) < 2:
        return 0.0
    s = sorted(vals)
    lo = s[max(0, int(0.05 * (len(s) - 1)))]
    hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
    return hi - lo


def _median(vals: Sequence[float]) -> float:
    if not vals:
        return 0.0
    s = sorted(vals)
    n = len(s)
    return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


def estimate_play_axis(points) -> Tuple[str, float, float]:
    """Return (axis 'x'|'y', net_value, span) from visible points.

    Play axis = the image axis along which the shuttle travels most (larger spread).
    For a baseline camera that's Y (players top/bottom); for a side camera, X.
    """
    xs = [p.x for p in points if p.visible]
    ys = [p.y for p in points if p.visible]
    sx, sy = _spread(xs), _spread(ys)
    if sx >= sy:
        return "x", _median(xs), sx
```

```
    return "y", _median(ys), sy


def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
    """Count sign changes of (coord - net) across visible points, ignoring the
    deadband zone near the net (so jitter at the net doesn't inflate the count)."""
    crossings = 0
    last_side = None
    for p in window_points:
        if not p.visible:
            continue
        v = (p.x if axis == "x" else p.y) - net
        if abs(v) < deadband:
            continue
        side = v > 0
        if last_side is not None and side != last_side:
            crossings += 1
        last_side = side
    return crossings


def _points_in(points, fps: float, start: float, end: float):
    return [p for p in points if start * fps <= p.frame <= end * fps]


def gate_windows(points, windows: List[Interval], fps: float,
                 min_crossings: int = 2, deadband_frac: float = 0.06,
                 estimate: Optional[Tuple[str, float, float]] = None) -> List[GatedWindow]:
    """Annotate each (start,end) window with its shuttle net-crossing count and a
    keep decision (crossings >= min_crossings). Deadband is a fraction of the play
    axis span. Net/axis are estimated once over the whole trajectory (stable).

    `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
    `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't redone
    per window ? same result, avoids the per-window re-estimation."""
    axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
    deadband = deadband_frac * span
    out: List[GatedWindow] = []
    for (s, e) in windows:
        wp = _points_in(points, fps, s, e)
        c = count_net_crossings(wp, axis, net, deadband)
        vis = sum(1 for p in wp if p.visible)
        out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                               kept=c >= min_crossings))
    return out


def apply_gate(points, windows: List[Interval], fps: float,
               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
    """Convenience: return only the windows that pass the crossing gate."""
    return [(g.start, g.end) for g in
            gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

20. assistant (2026-06-14T10:33:38.009Z)

rally_gate is pure (no numpy needed). `FusionSegmenter(None, {})` is instantiable. The two seams only read the passed-in `idx_cfg`. Now let me find the app's config loader so I reproduce production `idx_cfg` exactly (not hand-build it), and confirm cached golden trajectories + the score/label tooling exist.

21. user (2026-06-14T10:33:41.668Z)

```
=== config loader ===
__init__.py
__pycache__
loader.py
models.py
---
backend/config/loader.py:6:This replaces scattered json.load + .get() patterns throughout the
codebase.
backend/config/loader.py:50:    return AppConfig().model_dump(mode="json")
backend/config/loader.py:53:def load_config(path: str | Path | None = None) -> AppConfig:
backend/config/loader.py:69:                file_data = json.load(f)
backend/config/loader.py:100:    cfg_path.write_text(cfg.model_dump_json(indent=4),
encoding="utf-8")
backend/config/models.py:240:class AppConfig(BaseModel):
backend/config/models.py:269:    # via model_dump, so legacy chained access keeps working ?
returning the Pydantic
backend/config/models.py:289:            return val.model_dump(mode="json")

=== top-level config.json keys ===
['sport', 'ai_provider', 'ai_model', 'validation', 'indexing', 'output', 'stitching']

=== golden trajectories present? ===
adarsh_avi_singles     traj=OK labels=OK fps=59.94
kushagra_singles       traj=OK labels=OK fps=59.94
largetest_doubles      traj=OK labels=OK fps=29.97
gbaaddy                traj=OK labels=OK fps=59.94
testlarge_short        traj=OK labels=OK fps=29.97
mahadevpura_singles    traj=OK labels=OK fps=29.97
```

22. user (2026-06-14T10:33:44.839Z)

```
=== env: numpy/cv2/torch present? ===
numpy 2.4.6
cv2 4.13.0
=== load_golden_gts + score + label_candidates signatures ===
33:def load_golden_gts(csv_path: str) -> List[Interval]:
104:def score(preds: List[Interval], gts: List[Interval],
50:def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
39:def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
```

23. assistant (2026-06-14T10:33:53.155Z)

All data present (6/6 golden trajectories + labels), numpy/cv2 available. Let me read the config loader and the AppConfig dict-like access so I instantiate the segmenter exactly as production does.

24. user (2026-06-14T10:33:54.111Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
```

```python

from .models import AppConfig

logger = logging.getLogger(__name__)

DEFAULT_CONFIG_PATH = Path("config.json")


def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str = "") -> list[str]:
    """Dotted paths in `data` that the typed config will not honor.

    Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
    kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
    typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
    user's intent is lost ? collect every such key so load_config can warn.
    """
    unknown: list[str] = []
    fields = model_cls.model_fields
    for key, val in data.items():
        if key not in fields:
            unknown.append(prefix + key)
            continue
        ann = fields[key].annotation
        if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann, BaseModel):
            unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
    return unknown


def _get_builtin_defaults() -> dict[str, Any]:
    """Return a plain dict of the current built-in defaults.

    The AppConfig model is the single source of truth for defaults.
    """
    return AppConfig().model_dump(mode="json")


def load_config(path: str | Path | None = None) -> AppConfig:
    """Load configuration with the following precedence:

    1. Built-in defaults defined in the Pydantic models.
    2. Values from the JSON file (if present and readable).
    3. (Future) environment / CLI overrides.

    Missing file or unreadable file ? returns a fully defaulted AppConfig.
    Unknown keys in the file are tolerated (extra="allow" on the model).
    """
    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

    file_data: dict[str, Any] = {}
    if cfg_path.exists():
        try:
            with open(cfg_path, "r", encoding="utf-8") as f:
                file_data = json.load(f)
        except Exception as exc:
            # Non-fatal: continue with defaults + whatever we could parse.
            # In production we might log here.
            logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

    if file_data:
        unknown = _unknown_key_paths(file_data, AppConfig)
        if unknown:
            logger.warning(
```

```
                "[CONFIG WARNING] %s contains keys the typed config does not "
                "recognize (typo'd or removed knobs ? top-level extras are kept "
                "but unread; unknown section keys are silently dropped): %s",
                cfg_path, ", ".join(sorted(unknown)),
            )

    # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
    merged = {**_get_builtin_defaults(), **file_data}

    return AppConfig.model_validate(merged)


def save_default_config(path: str | Path | None = None) -> Path:
    """Write a fresh default config.json.

    Used by setup.py diagnostics and the --setup CLI path.
    Overwrites if the file exists.
    """
    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
    cfg = AppConfig()
    cfg_path.parent.mkdir(parents=True, exist_ok=True)
    cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
    return cfg_path


# Backwards-compatible alias used by some older call sites during transition.
def get_default_config() -> dict[str, Any]:
    """Return defaults as a plain dict (for transitional use only)."""
    return _get_builtin_defaults()

```

25. user (2026-06-14T10:33:54.793Z)

```
class AppConfig(BaseModel):
    """Root configuration object.

    All runtime configuration should be accessed via an instance of this class
    (or the loader) rather than raw dict lookups.
    """

    sport: Sport = "badminton"
    ai_provider: str = "gemini"
    # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable flagship;
    # the alias never deprecates underneath us. Eval reports record the run date alongside.
    ai_model: str = "gemini-flash-latest"

    validation: ValidationConfig = Field(default_factory=ValidationConfig)
    indexing: IndexingConfig = Field(default_factory=IndexingConfig)
    output: OutputConfig = Field(default_factory=OutputConfig)
    stitching: StitchingConfig = Field(default_factory=StitchingConfig)
    security: SecurityConfig = Field(default_factory=SecurityConfig)
    logging: LoggingConfig = Field(default_factory=LoggingConfig)

    # Allow extra keys during transition so existing config.json files do not break.
    model_config = {
        "extra": "allow",
        "validate_assignment": True,
    }

    # Temporary migration shim for legacy dict-style access (config.get("section", {}).get(...)).
    # Will be removed once all call sites (validators, segmenters, etc.) move to typed
    # attribute access / the typed config. A nested *section* is returned as a plain
```

```
dict
    # via model_dump, so legacy chained access keeps working ? returning the Pydantic
    # section model instead breaks it (a model has no .get), which bit every validator on
    # real runs. Scalar leaves are returned as-is.
    def get(self, key: str, default: Any = None) -> Any:
        _MISSING = object()
        val: Any = _MISSING
        if key in type(self).model_fields or hasattr(self, key):
            val = getattr(self, key, _MISSING)
        if val is _MISSING:
            # Legacy convenience: find a leaf key inside any nested section. Sections are
            # discovered dynamically from the model fields, so a future top-level section
            # (e.g. `audio`, `jobs`) is covered automatically without editing a hardcoded list.
            for fname in type(self).model_fields:
                section = getattr(self, fname, None)
                if isinstance(section, BaseModel) and hasattr(section, key):
                    val = getattr(section, key)
                    break
        if val is _MISSING:
            return default
        if isinstance(val, BaseModel):
            return val.model_dump(mode="json")
        return val

    def __getitem__(self, key: str) -> Any:
        val = self.get(key)
        if val is None and key not in type(self).model_fields:
            # Re-raise like a real dict for unknown top-level
            raise KeyError(key)
        return val


# Convenience type alias for callers that want to annotate
Config = AppConfig

```

26. assistant (2026-06-14T10:34:30.152Z)

Everything needed is present and I now know how to reproduce production exactly: `load_config()` ? `config.get("indexing")` is a plain dict, and the segmenter seams only read the passed-in `idx_cfg`. Let me confirm the runner's parse/windowing signatures and the `score` function so my harness calls them correctly.

27. user (2026-06-14T10:34:33.003Z)

```
=== parse_trajectory_csv + trajectory_to_action_windows signatures ===
233:    def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
252:    def trajectory_to_action_windows(

=== trajectory_to_action_windows full signature ===
    def trajectory_to_action_windows(
        points: List[TrajectoryPoint],
        fps: float,
        frame_width: float,
        chunk_start: float = 0.0,
        velocity_thresh: float = 0.02,
        merge_gap: float = 2.0,
        min_window_duration: float = 2.0,

=== metrics.score ===
def score(preds: List[Interval], gts: List[Interval],
```

```
          iou_threshold: float = 0.5,
          match_result: Optional[MatchResult] = None) -> Score:
    """Compute precision/recall/F1, mean matched IoU, and boundary errors.

    Pass a precomputed `match_result` to avoid re-matching (e.g. when also
    extracting the failure lists); otherwise matching is done here.
    """
    mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

    tp = len(mr.matches)
    fp = len(mr.unmatched_pred_indices)
    fn = len(mr.unmatched_gt_indices)

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0 else 0.0

    if mr.matches:
        mean_iou = sum(m.iou for m in mr.matches) / tp
        mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
        mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in mr.matches)
/ tp
    else:
        mean_iou = mean_start_err = mean_end_err = 0.0

    return Score(
        iou_threshold=iou_threshold,
        num_pred=len(preds), num_gt=len(gts),
        true_positives=tp, false_positives=fp, false_negatives=fn,
        precision=precision, recall=recall, f1=f1,
        mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
    )


def pr_curve(preds: List[Interval], gts: List[Interval],
             thresholds: Optional[List[float]] = None) -> List[Score]:
    """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).

    Lets you see strict-vs-lenient behaviour rather than a single operating point.
    """
    if thresholds is None:
        thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
    return [score(preds, gts, iou_threshold=t) for t in thresholds]

=== label_candidates ===
def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
                     iou_threshold: float = 0.5) -> List[int]:
    """Label each candidate window 1 (real rally) or 0 (spurious).

    A window is positive iff it is matched one-to-one to a ground-truth rally at
    `iou_threshold` ? the *same* matching the evaluator uses for true positives,
    so the classifier is trained on exactly what we score. Substrate-agnostic:
    works for YOLO or WASB-derived windows alike.
    """
    mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
    positive = {m.pred_index for m in mr.matches}
    return [1 if i in positive else 0 for i in range(len(candidate_intervals))]


def build_training_set(
    candidate_rows: List[Dict],
    gt_intervals: List[Interval],
    iou_threshold: float = 0.5,
```

```
    feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
) -> Tuple[List[List[float]], List[int], List[Interval]]:
    """Turn one video's candidate rows + ground truth into (X, y, intervals).

    `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
    features without changing the labeling logic. Returns the feature matrix `X`,
    binary labels `y`, and the candidate intervals (handy for downstream merging).
    """
    intervals: List[Interval] = [
        (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
    ]
    y = label_candidates(intervals, gt_intervals, iou_threshold)
    X = [feature_fn(r) for r in candidate_rows]
    return X, y, intervals
```

28. assistant (2026-06-14T10:35:12.153Z)

I have everything. Let me confirm `wasb_hybrid` inherits the base seams unchanged (so the mc=0 identity claim is real), then write the verification harness.

29. user (2026-06-14T10:35:12.968Z)

```
"""WASB-based hybrid play segmenter.

Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
badminton weights and runs today, so this is the live shuttle-motion substrate.

For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
min_window_duration) are what get tuned against golden rally labels.

The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
WASB runner and identity.
"""
from typing import Any, Dict, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


class WasbHybridSegmenter(TrajectoryHybridSegmenter):
    family = "wasb"
    display_label = "WASB"

    @property
    def name(self) -> str:
        return "wasb_hybrid"

    @property
    def description(self) -> str:
        return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained badminton "
                "weights) for precise candidate rallies + AI provider for semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        cfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(cfg), {"weights_path": cfg.weights_path}


```

```
    segmenter_registry.register(WasbHybridSegmenter)

```

30. assistant (2026-06-14T10:35:40.460Z)

Confirmed: `WasbHybridSegmenter` inherits both base seams unchanged, and at `min_crossings=0` the fusion windowing params equal wasb's ? so the mc=0 identity is real and I'll demonstrate it empirically rather than assert it. Now I'll write the served-side verification harness.

31. assistant (2026-06-14T10:36:23.412Z)

```
"""Served-side verification of the Gate-A default flip (PR #146).

Confirms the LIVE `FusionSegmenter` served path (not the offline harness's
recall-oriented proposal) behaves as the flip promises, WITHOUT re-running WASB
(wasb_runner has no cache-hit early-return ? re-running would overwrite the cached
CSV). We reuse each golden video's CACHED WASB trajectory and drive the REAL served
code: the same production windowing `process_video` uses, then the actual
`FusionSegmenter._enrich_candidate_stats` (net_crossings) and `_select_for_handoff`
(the served Gate-A gate) ? the exact methods Gemini's handoff set is built from.

Two questions (the owner-gated follow-up flagged in #146):
  (a) At indexing.fusion.min_crossings=0 (ungated control), does the fusion handoff
      set match a wasb_hybrid run on the same trajectory? (byte-identical seeding +
      handoff at mc=0 ? the no-regression invariant on the real served path.)
  (b) At min_crossings=2 (the Gate-A default), does it drop ONLY low-crossing
      (held-shuttle/feed) windows ? no real-rally loss vs golden labels ? i.e. no
      F1 regression?

Config is loaded EXACTLY as production (`load_config()`); segmenters come from the
registry. Honest measured numbers only. Artifacts isolated under
output/gateA_served_verify/. No GPU, no Gemini, no WASB re-run.
"""
from __future__ import annotations

import copy
import json
import os
from typing import Any, Dict, List, Tuple

from backend.config.loader import load_config
from backend.pipeline.segmenters.base import segmenter_registry
```

Import for side-effect registration of both segmenters.

```
import backend.pipeline.segmenters.fusion_hybrid  # noqa: F401
import backend.pipeline.segmenters.wasb_hybrid  # noqa: F401
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.eval.metrics import score

Interval = Tuple[float, float]
MANIFEST = "output/human_lovo_manifest.json"
OUT_DIR = "output/gateA_served_verify"
IOU = 0.5


def win_params(idx_cfg: Dict[str, Any], family: str) -> Tuple[float, float, float]:
    """Reproduce TrajectoryHybridSegmenter.process_video's windowing knobs exactly."""
    vt = idx_cfg.get(family, {}).get("velocity_threshold", 0.02)
    mg = idx_cfg.get("dead_time_merge_gap", 2.0)
    mwd = idx_cfg.get("min_action_window_duration", 2.0)
    return float(vt), float(mg), float(mwd)
```

```python
def windows_for(points, fps: float, fw: float, vt: float, mg: float, mwd: float) ->
List[Dict[str, Any]]:
    return TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=fw, chunk_start=0.0,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd, chunk_index=0,
    )


def ivs(windows: List[Dict[str, Any]], idxs) -> List[Interval]:
    return [(windows[i]["start"], windows[i]["end"]) for i in idxs]


def run_video(spec: Dict[str, Any], fusion, wasb, base_idx_cfg) -> Dict[str, Any]:
    name = spec["name"]
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    points = TrackNetRunner.parse_trajectory_csv(spec["trajectory"])
    gts = load_golden_gts(spec["labels"])

    # --- Served windowing for each segmenter, from its own family config ---
    f_vt, f_mg, f_mwd = win_params(base_idx_cfg, fusion.family)   # family="fusion"
    w_vt, w_mg, w_mwd = win_params(base_idx_cfg, wasb.family)     # family="wasb"
    f_windows = windows_for(points, fps, fw, f_vt, f_mg, f_mwd)
    w_windows = windows_for(points, fps, fw, w_vt, w_mg, w_mwd)

    # (a) Same seeding: fusion and wasb produce identical candidate windows.
    f_iv = [(w["start"], w["end"]) for w in f_windows]
    w_iv = [(w["start"], w["end"]) for w in w_windows]
    seeding_identical = f_iv == w_iv

    # Real served enrichment (net_crossings) via the actual FusionSegmenter method.
    f_stats = [fusion._enrich_candidate_stats(cw, points, fps, fw, "", base_idx_cfg)
               for cw in f_windows]

    # Handoff selection through the REAL served gate at mc=0 and mc=2.
    cfg0 = copy.deepcopy(base_idx_cfg); cfg0.setdefault("fusion", {})["min_crossings"] = 0
    cfg2 = copy.deepcopy(base_idx_cfg); cfg2.setdefault("fusion", {})["min_crossings"] = 2
    keep0 = fusion._select_for_handoff(f_windows, f_stats, cfg0)
    keep2 = fusion._select_for_handoff(f_windows, f_stats, cfg2)
    # wasb_hybrid's handoff = the base seam (all windows, no gating).
    wasb_keep = wasb._select_for_handoff(w_windows, [{} for _ in w_windows], base_idx_cfg)

    # (a) Handoff identity at mc=0: same windows AND same selected set as wasb_hybrid.
    handoff_identical = (seeding_identical and list(keep0) == list(range(len(f_windows)))
                         and list(wasb_keep) == list(range(len(w_windows))))

    # (b) Label every window once vs golden; classify each Gate-A drop as FP or real-rally loss.
    labels = label_candidates(f_iv, gts, IOU)
    dropped = [i for i in keep0 if i not in set(keep2)]
    drops = [{"start": round(f_windows[i]["start"], 2), "end": round(f_windows[i]["end"], 2),
              "net_crossings": f_stats[i].get("net_crossings", 0.0),
              "is_real_rally": bool(labels[i])} for i in dropped]
    real_rally_losses = [d for d in drops if d["is_real_rally"]]

    s0 = score(ivs(f_windows, keep0), gts, IOU)
    s2 = score(ivs(f_windows, keep2), gts, IOU)

    return {
        "name": name, "fps": fps,
        "num_gts": len(gts), "num_windows": len(f_windows),
        "seeding_identical_to_wasb": seeding_identical,
        "handoff_identical_at_mc0": handoff_identical,
        "win_params_fusion": [f_vt, f_mg, f_mwd], "win_params_wasb": [w_vt, w_mg, w_mwd],
        "handoff_mc0": len(keep0), "handoff_mc2": len(keep2),
        "dropped_by_gateA": len(dropped),
        "real_rally_losses": len(real_rally_losses),
```

```
        "drops": drops,
        "mc0": {"P": round(s0.precision, 3), "R": round(s0.recall, 3), "F1": round(s0.f1, 3),
                "TP": s0.true_positives, "FP": s0.false_positives, "FN": s0.false_negatives},
        "mc2": {"P": round(s2.precision, 3), "R": round(s2.recall, 3), "F1": round(s2.f1, 3),
                "TP": s2.true_positives, "FP": s2.false_positives, "FN": s2.false_negatives},
        "delta_f1": round(s2.f1 - s0.f1, 3),
    }


def main() -> None:
    os.makedirs(OUT_DIR, exist_ok=True)
    config = load_config()                      # exactly production
    idx_cfg = config.get("indexing", {})        # plain dict, identical to served path
    FusionCls = segmenter_registry.get("fusion_hybrid")
    WasbCls = segmenter_registry.get("wasb_hybrid")
    fusion = FusionCls(None, config)
    wasb = WasbCls(None, config)

    specs = json.load(open(MANIFEST, encoding="utf-8"))
    print("=" * 92)
    print("SERVED-SIDE Gate-A verification ? live FusionSegmenter over CACHED golden
trajectories")
    print(f"(config.indexing.fusion = {idx_cfg.get('fusion')})")
    print("=" * 92)

    results = [run_video(s, fusion, wasb, idx_cfg) for s in specs]

    print(f"\n{'video':<22}{'win':>4}{'mc0':>5}{'mc2':>5}{'drop':>5}{'lost':>5}"
          f"{'F1@mc0':>8}{'F1@mc2':>8}{'?F1':>7}  seed=wasb  handoff==wasb@mc0")
    all_identical = True
    total_lost = 0
    for r in results:
        all_identical &= r["seeding_identical_to_wasb"] and r["handoff_identical_at_mc0"]
        total_lost += r["real_rally_losses"]
        print(f"{r['name']:<22}{r['num_windows']:>4}{r['handoff_mc0']:>5}{r['handoff_mc2']:>5}"
              f"{r['dropped_by_gateA']:>5}{r['real_rally_losses']:>5}"
              f"{r['mc0']['F1']:>8.3f}{r['mc2']['F1']:>8.3f}{r['delta_f1']:>+7.3f}"
              f"   {str(r['seeding_identical_to_wasb']):>5}
{str(r['handoff_identical_at_mc0']):>5}")

    n = len(results)
    mean_f1_0 = sum(r["mc0"]["F1"] for r in results) / n
    mean_f1_2 = sum(r["mc2"]["F1"] for r in results) / n
    mean_p0 = sum(r["mc0"]["P"] for r in results) / n
    mean_p2 = sum(r["mc2"]["P"] for r in results) / n
    mean_r0 = sum(r["mc0"]["R"] for r in results) / n
    mean_r2 = sum(r["mc2"]["R"] for r in results) / n

    print("-" * 92)
    print(f"MEAN over n={n}:  P {mean_p0:.3f}->{mean_p2:.3f}   R {mean_r0:.3f}->{mean_r2:.3f}
"
          f"F1 {mean_f1_0:.3f}->{mean_f1_2:.3f}  (? {mean_f1_2 - mean_f1_0:+.3f})")
    print(f"\n(a) mc=0 seeding+handoff byte-identical to wasb_hybrid on all videos : "
          f"{'YES ?' if all_identical else 'NO ?'}")
    print(f"(b) real-rally windows dropped by Gate-A (mc=2) across corpus        : "
          f"{total_lost}  {'(none ? drops are held-shuttle/feed FPs) ?' if total_lost == 0 else
'?'}")
    print(f"    F1 regression on any video                                       : "
          f"{'NONE ?' if all(r['delta_f1'] >= 0 for r in results) else 'YES ?'}")

    summary = {
        "purpose": "served-side Gate-A flip verification (PR #146) via live FusionSegmenter "
                   "over cached golden trajectories ? no WASB re-run, no Gemini",
        "config_fusion": idx_cfg.get("fusion"),
        "iou": IOU, "n": n,
```

```
        "mc0_seeding_handoff_identical_to_wasb_all_videos": all_identical,
        "total_real_rally_losses_gateA": total_lost,
        "no_f1_regression_any_video": all(r["delta_f1"] >= 0 for r in results),
        "mean_f1_mc0": round(mean_f1_0, 3), "mean_f1_mc2": round(mean_f1_2, 3),
        "mean_delta_f1": round(mean_f1_2 - mean_f1_0, 3),
        "per_video": results,
    }
    json.dump(summary, open(os.path.join(OUT_DIR, "served_gateA_summary.json"), "w"), indent=2)
    print(f"\nwrote {OUT_DIR}/served_gateA_summary.json")


if __name__ == "__main__":
    main()
```

32. user (2026-06-14T10:36:23.442Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\served_gateA_verify.py (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-14T10:36:27.158Z)

Now let me run the served-side verification harness.

34. user (2026-06-14T10:36:34.735Z)

```
============================================================================================
SERVED-SIDE Gate-A verification ? live FusionSegmenter over CACHED golden trajectories
(config.indexing.fusion = {'substrate': 'wasb', 'velocity_threshold': 0.02, 'cue_flags': {},
'min_crossings': 2, 'min_players': 0, 'court_polygon': None, 'net_axis': 'y', 'net_position':
0.5})
============================================================================================

video                  win  mc0  mc2 drop lost  F1@mc0  F1@mc2    ?F1  seed=wasb
handoff==wasb@mc0
adarsh_avi_singles      97   97   76   21    7   0.632   0.628 -0.004    True       True
kushagra_singles         2    2    2    0    0   0.000   0.000 +0.000    True       True
largetest_doubles       52   52   42   10    5   0.535   0.484 -0.051    True       True
gbaaddy                 22   22   20    2    0   0.086   0.088 +0.003    True       True
testlarge_short          5    5    5    0    0   0.182   0.182 +0.000    True       True
mahadevpura_singles     24   24   23    1    0   0.364   0.370 +0.007    True       True
--------------------------------------------------------------------------------------------
MEAN over n=6:  P 0.317->0.337   R 0.290->0.261   F1 0.300->0.292  (? -0.008)

(a) mc=0 seeding+handoff byte-identical to wasb_hybrid on all videos : YES ?
(b) real-rally windows dropped by Gate-A (mc=2) across corpus       : 12  ?
    F1 regression on any video                                      : YES ?

wrote output/gateA_served_verify/served_gateA_summary.json
```

35. assistant (2026-06-14T10:37:32.395Z)

Important ? this is a substantive finding, not a rubber-stamp. Let me read it carefully:

- **(a) holds**: at `mc=0` the fusion seeding + handoff is byte-identical to `wasb_hybrid` on all 6 videos. ? The no-regression invariant on the real served path is confirmed.
- **(b) does NOT reproduce the offline win**: on the **production** windowing, Gate-A drops **12 windows that actually match golden rallies** (real-rally loss), mean F1 goes 0.300?0.292 (**?0.008**), with largetest_doubles ?0.051.

The offline `+0.074` was measured on the recall-oriented *proposal* windowing `(0.005,1.0,0.5)`; production serves a coarse windowing `(0.02,2.0,2.0)`. I must verify this is a real windowing-driven effect and not a harness artifact before recording it. Let me re-run the offline LOVO baseline and inspect the dropped real-rally windows.

36. user (2026-06-14T10:37:38.382Z)

```
=== Re-run offline ab_n6_lovo (proposal windowing) to confirm +0.074 baseline ===
================================================================================
FIRST A/B EXPERIMENT ? n=6 golden corpus (offline, persisted trajectories)
================================================================================
  adarsh_avi_singles      172 candidates   96 rallies  proposal-recall-ceiling=0.833
  kushagra_singles         20 candidates   32 rallies  proposal-recall-ceiling=0.125
  largetest_doubles        86 candidates   49 rallies  proposal-recall-ceiling=0.816
  gbaaddy                  82 candidates   48 rallies  proposal-recall-ceiling=0.312
  testlarge_short          14 candidates    6 rallies  proposal-recall-ceiling=0.500
  mahadevpura_singles      57 candidates   31 rallies  proposal-recall-ceiling=0.645

--- per-video held-out F1 ---
  video                   CONTROL    GateA  learned
  adarsh_avi_singles        0.597    0.722    0.613
  kushagra_singles          0.154    0.163    0.163
  largetest_doubles         0.593    0.667    0.535
  gbaaddy                   0.231    0.274    0.222
  testlarge_short           0.300    0.375    0.000
  mahadevpura_singles       0.455    0.571    0.308

--- A/B 1: CONTROL vs Gate-A (static) ? C1 honest aggregation ---
  meanF1 0.388 -> 0.462  ?F1=+0.074 CI[0.042, 0.104] excl0=True  sign={'wins': 6, 'losses': 0,
'ties': 0, 'n_effective': 6, 'p_value': 0.031}
--- A/B 2: CONTROL vs learned fusion (LOVO) ? C1 honest aggregation ---
  meanF1 0.388 -> 0.307  ?F1=-0.081 CI[-0.173, -0.002] excl0=True  sign={'wins': 2, 'losses': 4,
'ties': 0, 'n_effective': 6, 'p_value': 0.688}

--- C4 over-segmentation-aware metrics (mean across videos) ---
  CONTROL   {'f1@10': 0.645, 'f1@25': 0.603, 'f1@50': 0.388, 'seg_count_ratio': 1.68}
  gateA     {'f1@10': 0.748, 'f1@25': 0.698, 'f1@50': 0.462, 'seg_count_ratio': 1.01}
  learned   {'f1@10': 0.564, 'f1@25': 0.509, 'f1@50': 0.307, 'seg_count_ratio': 0.89}
  learned out-of-fold (C3) mAP@tIoU[.3-.7] = 0.245

wrote output/ab_experiment_n6.json
```

37. user (2026-06-14T10:37:38.590Z)

```
=== Dropped real-rally windows (net_crossings) on production windowing ===
adarsh_avi_singles     lost= 7  net_crossings of lost real rallies: [1.0, 0.0, 1.0, 1.0, 1.0,
1.0, 1.0]
                       dropped-FPs=14 (correct drops)  total-windows=97 gts=96
                       dropped-FPs=0 (correct drops)  total-windows=2 gts=32
largetest_doubles      lost= 5  net_crossings of lost real rallies: [1.0, 1.0, 1.0, 1.0, 1.0]
                       dropped-FPs=5 (correct drops)  total-windows=52 gts=49
                       dropped-FPs=2 (correct drops)  total-windows=22 gts=48
                       dropped-FPs=0 (correct drops)  total-windows=5 gts=6
                       dropped-FPs=1 (correct drops)  total-windows=24 gts=31
```

38. assistant (2026-06-14T10:38:39.393Z)

Now the picture is rigorous and clear. Let me state what the two runs establish:

- **Offline harness reproduced exactly**: proposal windowing `(0.005,1.0,0.5)`, CONTROL F1 0.388 ? Gate-A 0.462, **+0.074**, 6/6, p=0.031, seg_count_ratio **1.68?1.01** (it was cleaning up heavy over-segmentation).
- **Served path**: production windowing `(0.02,2.0,2.0)` already merges aggressively (low over-segmentation), so Gate-A has little FP to remove and instead **drops 12 real-rally windows whose net_crossings are 0?1** (almost all exactly 1) ? mean F1 **?0.008**, worst on doubles.

The lost rallies sit at `net_crossings=1`, which suggests `min_crossings=1` might be the right served operating point. Let me sweep `min_crossings ? {0,1,2,3}` on the production windowing to give the owner the operating-point curve ? that's directly decision-relevant.

39. assistant (2026-06-14T10:38:57.768Z)

```
"""Operating-point sweep for Gate-A on the SERVED (production) windowing.

served_gateA_verify.py showed the Gate-A default (min_crossings=2) drops real-rally
windows on the production windowing because their WASB trajectory registered only
0-1 net crossings. This sweeps min_crossings to find the served operating point:
for each threshold, mean F1 + total real-rally windows lost vs the mc=0 control.
Reuses the same live FusionSegmenter seams. No GPU / Gemini / WASB re-run.
"""
from __future__ import annotations

import copy
import json
from typing import Any, Dict, List

from backend.config.loader import load_config
from backend.pipeline.segmenters.base import segmenter_registry
import backend.pipeline.segmenters.fusion_hybrid  # noqa: F401
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.eval.metrics import score

MANIFEST = "output/human_lovo_manifest.json"
IOU = 0.5
THRESHOLDS = [0, 1, 2, 3]


def main() -> None:
    config = load_config()
    idx_cfg = config.get("indexing", {})
    fusion = segmenter_registry.get("fusion_hybrid")(None, config)
    specs = json.load(open(MANIFEST, encoding="utf-8"))

    vt = idx_cfg.get(fusion.family, {}).get("velocity_threshold", 0.02)
    mg = idx_cfg.get("dead_time_merge_gap", 2.0)
    mwd = idx_cfg.get("min_action_window_duration", 2.0)

    per_video: List[Dict[str, Any]] = []
    for s in specs:
        fps, fw = float(s["fps"]), float(s["frame_width"])
        points = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
        gts = load_golden_gts(s["labels"])
        windows = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=fw, chunk_start=0.0,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd, chunk_index=0)
        stats = [fusion._enrich_candidate_stats(cw, points, fps, fw, "", idx_cfg) for cw in
windows]
        iv = [(w["start"], w["end"]) for w in windows]
        labels = label_candidates(iv, gts, IOU)
        row: Dict[str, Any] = {"name": s["name"], "n_win": len(windows), "n_gts": len(gts)}
        base_keep = None
        for mc in THRESHOLDS:
            cfg = copy.deepcopy(idx_cfg); cfg.setdefault("fusion", {})["min_crossings"] = mc
            keep = fusion._select_for_handoff(windows, stats, cfg)
            if mc == 0:
                base_keep = set(keep)
            sc = score([iv[i] for i in keep], gts, IOU)
            lost = sum(1 for i in base_keep if i not in set(keep) and labels[i] == 1)
            row[f"mc{mc}"] = {"kept": len(keep), "F1": round(sc.f1, 3),
                              "P": round(sc.precision, 3), "R": round(sc.recall, 3),
                              "real_lost_vs_mc0": lost}
        per_video.append(row)
```

```
    n = len(per_video)
    print("=" * 88)
    print("Gate-A operating-point sweep on the SERVED (production) windowing  "
          f"(vt={vt}, merge_gap={mg}, min_win={mwd})")
    print("=" * 88)
    print(f"{'metric':<20}" + "".join(f"{'mc='+str(mc):>14}" for mc in THRESHOLDS))
    mean_f1 = {mc: sum(v[f'mc{mc}']['F1'] for v in per_video) / n for mc in THRESHOLDS}
    mean_p = {mc: sum(v[f'mc{mc}']['P'] for v in per_video) / n for mc in THRESHOLDS}
    mean_r = {mc: sum(v[f'mc{mc}']['R'] for v in per_video) / n for mc in THRESHOLDS}
    tot_lost = {mc: sum(v[f'mc{mc}']['real_lost_vs_mc0'] for v in per_video) for mc in
THRESHOLDS}
    print(f"{'mean F1':<20}" + "".join(f"{mean_f1[mc]:>14.3f}" for mc in THRESHOLDS))
    print(f"{'mean precision':<20}" + "".join(f"{mean_p[mc]:>14.3f}" for mc in THRESHOLDS))
    print(f"{'mean recall':<20}" + "".join(f"{mean_r[mc]:>14.3f}" for mc in THRESHOLDS))
    print(f"{'real rallies lost':<20}" + "".join(f"{tot_lost[mc]:>14d}" for mc in THRESHOLDS))
    print(f"{'?F1 vs mc=0':<20}" + "".join(f"{mean_f1[mc]-mean_f1[0]:>+14.3f}" for mc in
THRESHOLDS))
    print("\n(mc=2 is the current served default from PR #146; mc=0 = Gate-A OFF =
wasb_hybrid)")


if __name__ == "__main__":
    main()
```

40. user (2026-06-14T10:38:57.804Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\served_gateA_sweep.py (file state is current in your context ? no need to Read
it back)
```

41. user (2026-06-14T10:39:15.591Z)

```
========================================================================================
Gate-A operating-point sweep on the SERVED (production) windowing  (vt=0.02, merge_gap=2.0,
min_win=2.0)
========================================================================================
metric                         mc=0          mc=1          mc=2          mc=3
mean F1                       0.300         0.303         0.292         0.269
mean precision                0.317         0.327         0.337         0.338
mean recall                   0.290         0.288         0.261         0.226
real rallies lost                 0             1            12            28
?F1 vs mc=0                  +0.000        +0.003        -0.008        -0.031

(mc=2 is the current served default from PR #146; mc=0 = Gate-A OFF = wasb_hybrid)
```

42. assistant (2026-06-14T10:39:40.446Z)

The operating-point curve is decisive and decision-relevant. On the **served** (production) windowing:

	mc=0 (off)	mc=1	**mc=2 (shipped)**	mc=3
mean F1	0.300	**0.303**	0.292	0.269
precision	0.317	0.327	0.337	0.338
recall	0.290	0.288	0.261	0.226
real rallies lost	0	1	12	28
?F1 vs off	?	+0.003	**?0.008**	?0.031

So at the served windowing, `mc=2` is mildly net-negative and drops 12 real rallies; `mc=1` would be the better operating point if Gate-A is kept on. Let me record this honestly in `docs/QUALITY_ITERATIONS.md`. Let me read its structure first.

43. user (2026-06-14T10:39:42.675Z)

```
=== file size + headings ===
1:# Rally-detection quality iterations ? living log
25:## Baselines that drive strategy (IoU 0.5 vs human golden labels)
41:## Current track (live) ? newest first
43:### PERSONALIZATION Phase-0 quality checkpoint ? the eval/safety RAILS for the per-user
feature *(2026-06-14)*
52:  promote noise and can't disable a structural guard (the proven Gate-A held-shuttle guard
stays on regardless of a thin
70:**Gate-A-only structural + `min_n = 5`**.
72:### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising
(not promotable) *(2026-06-14)*
91:### Gate-A flipped to the SERVED DEFAULT (owner-approved) *(2026-06-14)*
92:**Gate-A (net-crossings ? 2) is now the DEFAULT detection behaviour.** The owner approved
promoting the
95:- `backend/config/models.py` ? `FusionConfig.min_crossings` default **0 ? 2** (Gate A on
wherever
103:Gate A (ungated handoff, reproduces the substrate byte-for-byte), or
`indexing.default_segmenter: wasb_hybrid`
116:### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*
117:**Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The measured
win
120:bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the
eval-only Gate A
125:CONTROL 45 rallies vs **Gate-A 27**; Gate-A dropped **18 low-crossing windows (~1.7 min)**
of likely
135:**Next:** a real fusion-path run to confirm Gate-A served-side; per-video output layout
138:### First end-to-end A/B on the golden corpus (n=6) ? **ARCHIVED** *(2026-06-14)*
140:**Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test **p =
0.031**, CI
146:Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure is
the C2 (calibration)
149:### Eval-honesty helpers (course corrections C1?C4) ? additive, default-OFF *(2026-06-14)*
173:### Multi-signal fusion P2: the FusionSegmenter *(in progress, 2026-06-14)*
174:**Hypothesis.** Adjudicating shuttle-seeded windows with the net-crossing **Gate A**
(`min_crossings?2`)
188:**Data support for Gate A.** The same archive shows real rallies on the multi-court matches
contain
192:**Shipping shape.** A NEW `fusion` segmenter, **default OFF** (`default_segmenter`
unchanged); Gate A
210:Gate-A-pruned windows (a P3 redesign, would change the persisted-superset semantics); and
`fusion.velocity_threshold`
215:### Person cue extracted to a reusable producer (fusion P1) ?
[#115](https://github.com/avidullu/badminton-highlight-indexer/pull/115), 2026-06-13
221:### Experiment harness: A/B without rollback risk ?
[#114](https://github.com/avidullu/badminton-highlight-indexer/pull/114), 2026-06-13
231:## Archived predecessors (the historical trail)
245:## Standing lessons (carry-forward)
258:  Gate A leans on this.
264:- **Gate A already existed, just wasn't wired to production** ? the cheapest precision win
is plumbing, not
---- total lines ----
265 docs/QUALITY_ITERATIONS.md
```

44. assistant (2026-06-14T10:39:47.163Z)

Let me read the existing Gate-A entries to match style and place my entry correctly.

45. user (2026-06-14T10:39:48.034Z)

```
41      ## Current track (live) ? newest first
42
43      ### PERSONALIZATION Phase-0 quality checkpoint ? the eval/safety RAILS for the per-user
feature *(2026-06-14)*
44      The owner adopted the **HYBRID** posture: personalization as a **flag-gated FEATURE on a
```

generalization-first baseline**, paired with a **contribution flywheel** so per-user gains and the shared baseline compound together (full design in PERSONALIZATION_PLAN.md, merged [#144](https://github.com/avidullu/badminton-highlight-indexer/pull/144)). This checkpoint landed the three **honest-eval RAILS** that keep a per-user/small-N tuning layer from promoting noise or silently regressing a user:

- **Per-user promotion gate ? [#141](https://github.com/avidullu/badminton-highlight-indexer/pull/141)** (`backend/eval/per_user_gate.py`): a **min-N floor** + **structural-guard exemption** ? per-user/small-N tuning can't promote noise and can't disable a structural guard (the proven Gate-A held-shuttle guard stays on regardless of a thin user corpus).
- **Held-out-USER learning curve ? [#142](https://github.com/avidullu/badminton-highlight-indexer/pull/142)** (`backend/eval/per_user_eval.py`): turns the vague "<K videos to superior quality" claim into a **falsifiable release number** (the `smallest_k_with_lift` measured on held-out users).
- **Per-user no-regression guard ? [#143](https://github.com/avidullu/badminton-highlight-indexer/pull/143)**: a retrain/baseline-upgrade can **never make a user worse than their incumbent** (frozen per-user CONTROL; a regressing candidate is refused).

Read HONESTLY: these are the **eval/safety rails** for the personalization feature ? **not** new measured rally-detection wins. No F1 moved this checkpoint; what landed is the discipline that lets per-user tuning ship without regressing the baseline or over-claiming on n?6-per-user data.

Locked owner decisions (2026-06-14): identity = *design for the north-star (multi-user/cloud), implement for current use (single-user local)*; the inline per-user classifier sits **behind a cloud-load interface** (local store = one impl); **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate** (a one-video user is ALWAYS fully segmented by the baseline batteries); **free-tier videos ARE pooled** into the shared baseline (consent baked into free terms, minors-exclusion enforced at the pooling query); and the previously-pending structural-guard sub-decision is now LOCKED ? **Gate-A-only structural + `min_n = 5`**.

Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising (not promotable) *(2026-06-14)*

The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ? `output/*_golden_features.json`, the replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats (C1 ? bootstrap 95% CI + paired sign test; never a bare mean at n=6)**:

Variant (honest LOVO, IoU ? 0.5)	F1	?F1 vs prior	95% CI	sign test	CI clears 0
shuttle-only	0.307	?	?	?	?
+ person (P3)	0.286	**?0.021** (?6.7%)	[?0.165, +0.162]	2W/4L (p=0.688)	**No**
+ person + audio (P4 incr.)	0.366	**+0.079** (+27.7%)	[?0.054, +0.203]	4W/2L (p=0.688)	**No**

Read honestly: neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ?

`players_in_court` counts spectators); per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead ? +0.079 / +27.7%, wins 4/6 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above** shuttle-only (0.307) ? but n=6 can't confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and measure audio on shuttle-only *directly* (without the person handicap). Recorded in `eval_baselines/experiment_leaderboard.json`. **Negative/uncertain results are kept on purpose ? this is the honest ablation backbone for the paper aim.**

Gate-A flipped to the SERVED DEFAULT (owner-approved) *(2026-06-14)*

Gate-A (net-crossings ? 2) is now the DEFAULT detection behaviour. The owner approved promoting the validated held-shuttle guard from "recommended config" to the live served default. The flip is **two minimal, config-reversible changes**:

- `backend/config/models.py` ? `FusionConfig.min_crossings` default **0 ? 2** (Gate A on wherever `fusion_hybrid` serves).
- `config.json` ? `indexing.default_segmenter` **`gemini` ? `fusion_hybrid`** (+ an explicit `indexing.fusion` block making the served operating point visible). With default config, `FusionSegmenter` reproduces the WASB substrate's seeding/handoff exactly *except* that sub-2-crossing windows are dropped from the AI handoff ? the held-shuttle / one-sided-feed / warm-up false-positive fix.

Reversible ? rollback is a single flag (no code change): set `indexing.fusion.min_crossings: 0` to disable Gate A (ungated handoff, reproduces the substrate byte-for-byte), or `indexing.default_segmenter: wasb_hybrid` to revert the served segmenter entirely. **Gate B stays OFF** (`min_players = 0`, person cue OFF). **The persisted candidate superset is unchanged** ? gating filters only the AI handoff, so the offline harness can still re-score any variant; the offline ablation harness is untouched (it sets `Variant.min_crossings` explicitly and never reads `FusionConfig`). The validation evidence is unchanged from the promotion below (?F1 +0.074, 6/6, p=0.031, CI [+0.042, +0.105], over-seg 1.68?1.01) plus the GX010125 45?27 corroboration. **Recorded:** a `promotion` record appended to `eval_baselines/experiment_leaderboard.json`. ? **Honesty boundary:** the offline ?F1 is on the recall-oriented *proposal* candidate set, and the production fusion segmenter's served path is still only mock-tested on real video (P2). The gate's effect is **byte-reversible**, so this ships behind a one-flag rollback; a golden *served* regression on the fusion path remains a worthwhile optional follow-up (owner-gated GPU+Gemini run) to confirm no served-side regression, but it does not block the flip ? the offline 6/6 + p=0.031 ablation and the unit tests are the gate.

Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125 corroboration *(2026-06-14)*

Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness). The measured win (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg 1.68?1.01 ? the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md)) clears the honest bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the eval-only Gate A folded into production, MULTI_SIGNAL_FUSION_PLAN §3.1). ? The offline win is on the

```
recall-oriented *proposal*
122     candidate set; flipping the **live served default** stayed owner-gated at the time of
this entry ? **now resolved:
123     the owner approved the flip; see the SERVED-DEFAULT entry above.** **Corroborated on
real footage:** offline
124     re-score of GX010125_1080p's *cached* trajectory (no GPU, isolated
`output/GX010125_run/`) ? 45 candidates ?
125     CONTROL 45 rallies vs **Gate-A 27**; Gate-A dropped **18 low-crossing windows (~1.7
min)** of likely
126     held-shuttle/feed FPs (no golden labels for this footage ? divergence, not F1).
127
128     **Learned variant FIXED ? [#130](https://github.com/avidullu/badminton-highlight-
indexer/pull/130).** The
129     first A/B's learned logistic lost to CONTROL and zeroed `testlarge` ? a fixed-0.5-cutoff
artifact. Added
130     additive/default-OFF `Variant.tune_threshold` (operating point chosen on the **train**
fold) + `calibrate`
131     (Platt/isotonic, C2). Validated on the n=6 golden set: **mean F1 0.307 ? 0.423
(+0.116)**, testlarge
132     0.000 ? 0.300, and the learned variant now **beats CONTROL (0.388)**. Threshold-in-LOVO
is the big lever;
133     calibration is available too.
134
135     **Next:** a real fusion-path run to confirm Gate-A served-side; per-video output layout
136     ([PER_VIDEO_OUTPUT_LAYOUT.md](PER_VIDEO_OUTPUT_LAYOUT.md)) so each video's artifacts are
self-contained.
137
138     ### First end-to-end A/B on the golden corpus (n=6) ? **ARCHIVED** *(2026-06-14)*
139     First real use of the harness + C1?C4 helpers, 100% offline (re-scored persisted WASB
trajectories, no GPU).
140     **Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test
**p = 0.031**, CI
141     [+0.042, +0.104], over-segmentation (segment-count ratio) **1.68 ? 1.01**. The window-
level 5-feature logistic
142     **did not clear the bar** (?0.081, 2W/4L; a threshold/calibration artifact ? *not* the
stronger per-frame
143     prototype). The framework held up: **C1 promoted one variant and refused another; C4
surfaced over-segmentation
144     tIoU hid.** Full record + tables ?
145     [`archives/quality-iterations/2026-06-first-ab-experiment/`](archives/quality-
iterations/2026-06-first-ab-experiment/README.md).
146     Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure
is the C2 (calibration)
147     + threshold-in-loop task list.
148
149     ### Eval-honesty helpers (course corrections C1?C4) ? additive, default-OFF
*(2026-06-14)*
150     **Why.** A multi-agent literature review (folded into
151     [ANALYZERS_AND_RECONCILERS.md](ANALYZERS_AND_RECONCILERS.md) §13) confirmed the
framework's *skeleton* is
152     standard prior art (Temporal Action Localization/Segmentation + late/score-level fusion
? cite, don't claim),
153     and flagged that our **measurement** is walking into known small-N traps that would make
"promote only on
154     measured lift" promote noise or hide a real win. Four corrections, shipped as **pure,
additive helpers** (no
155     existing `compare`/LOVO/`metrics.score` behaviour changed ? callers opt in):
```

46. assistant (2026-06-14T10:40:28.876Z)

The existing flip entry (lines 110?114) explicitly anticipated this exact follow-up: *"the offline ?F1 is on the recall-oriented proposal candidate set? a golden served regression on the fusion path remains a worthwhile optional follow-up."* My run answers it. Let me record the honest result at the top of the live track.

47. user (2026-06-14T10:40:53.416Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md has been updated successfully. (file state is current in your context ? no need to Read it back)

48. assistant (2026-06-14T10:41:00.272Z)

Now let me update the live memory checkpoint with this finding.

49. user (2026-06-14T10:41:15.489Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

50. user (2026-06-14T10:41:21.810Z)

```
---
name: current-state
description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not only on request). What just landed, what's next, open decisions."
metadata:
  node_type: memory
  type: project
  originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
---

# Current state ? updated continuously (see [[working-agreement-checkpoints]])

**Checkpoint:** 2026-06-14 (PERSONALIZATION Phase-0 LANDED + quality checkpoint). **Owner adopted the HYBRID** ? personalization as a
```

51. user (2026-06-14T10:41:38.079Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

52. assistant (2026-06-14T10:41:43.033Z)

Recorded. Let me verify the working-tree state (I have not touched `config.json` ? the rollback is owner-gated).

53. user (2026-06-14T10:41:44.696Z)

```
 M docs/QUALITY_ITERATIONS.md
?? artifacts/
--- confirm config.json untouched (still mc=2) ---
min_crossings = 2
```

54. assistant (2026-06-14T10:42:18.692Z)

Verification complete and recorded. Only `docs/QUALITY_ITERATIONS.md` is modified; `config.json` is untouched (still `mc=2`), scratch scripts and artifacts are in gitignored dirs. Here's what I found.

Served-side Gate-A verification ? result

I drove the **live `FusionSegmenter` served seams** (`_enrich_candidate_stats` ? `net_crossings`, `_select_for_handoff` ? the gate) over each golden video's **cached** WASB trajectory, at the **production windowing** `process_video` actually uses (`vt=0.02, merge_gap=2.0, min_win=2.0`, loaded via `load_config()`). No WASB re-run, no GPU, no Gemini. Repro: [served_gateA_verify.py](scratch/served_gateA_verify.py) + [served_gateA_sweep.py](scratch/served_gateA_sweep.py).

(a) No-regression invariant ? CONFIRMED ?. At `min_crossings=0`, `fusion_hybrid`'s seeding **and** handoff set are **byte-identical to `wasb_hybrid` on all 6 golden videos**. The "fusion adds nothing when the gates are off" property is now verified on the *live* served code, not just by #117's review.

(b) The `+0.074` win does NOT transfer to the served path ??. That win was measured on the eval harness's recall-oriented **proposal** windowing `(0.005,1.0,0.5)`, where CONTROL over-segments (seg-ratio 1.68) and Gate-A's job is cleaning up fragmentation (?1.01). Production serves a **coarse** windowing that already merges aggressively ? so Gate-A has little FP to remove and instead trims **real rallies** whose WASB trajectory registered only 0?1 net crossings:

`min_crossings`	mean F1	precision	recall	real-rally windows lost	?F1
0 (off = `wasb_hybrid`)	0.300	0.317	0.290	0	?
1	**0.303**	0.327	0.288	1	**+0.003**
2 (shipped #146)	0.292	0.337	0.261	**12**	**?0.008**
3	0.269	0.338	0.226	28	?0.031

The shipped `mc=2` is a small mean-F1 **regression (?0.008)** and drops **12 windows that match golden rallies** (worst on doubles, `largetest_doubles ?0.051`). Honest caveats: deltas are **within n=6 noise** (not catastrophic), but `mc=2` is *not* a served improvement ? and because a dropped candidate **never reaches Gemini**, each lost rally is an **unrecoverable served recall loss**, while the precision "gain" partly duplicates what Gemini already does at the handoff. So the served tradeoff is **? neutral**.

Root cause: eval-harness windowing ? served windowing. Gate-A's value is conditional on over-segmentation; the coarse served windowing doesn't over-segment, so the gate inverts.

This is exactly the served-side follow-up the flip entry said was owed ? it doesn't unwind the merge, but the served numbers don't support `mc=2` as the default. The decision on what to do with the just-merged default is yours:

55. user (2026-06-14T10:42:30.440Z)

The user did not answer the questions.

56. assistant (2026-06-14T10:42:54.148Z)

I'll proceed with the safe default: no unilateral change to a production quality-default, and no commit without your go-ahead. Here's where things stand.

Done

- **Served-side verification ran** (offline, reusing cached golden trajectories ? no WASB re-run, no GPU, no Gemini).
- **(a) Confirmed**: at `mc=0`, `fusion_hybrid` is byte-identical to `wasb_hybrid` (seeding + handoff) on all 6 golden videos ? the no-regression invariant holds on the live served code.
- **(b) Surfaced honestly**: the offline `+0.074` win does **not** transfer to the served (production) windowing; the shipped `mc=2` is a within-noise mean-F1 regression (?0.008) that drops 12 real-rally windows. `mc=1` is the better served operating point.
- **Recorded** in docs/QUALITY_ITERATIONS.md (top of live track) and in the memory checkpoint (durable regardless of commit).

Working-tree state (nothing pushed, nothing committed)

- `docs/QUALITY_ITERATIONS.md` ? **modified, uncommitted** (the honest record).
- `config.json` ? **untouched**, still `min_crossings: 2`. I did not flip the default; that's your call.
- Repro scripts (`scratch/served_gateA_{verify,sweep}.py`) + artifacts (`output/gateA_served_verify/`) ? in gitignored dirs.

Your call (whenever ? not blocking)

The single-flag options are in the question above; my recommendation is **`mc=1`** (keeps the held-shuttle guard's intent ? drops 0-crossing dead feeds ? without over-dropping real

1-crossing rallies). The deeper fix worth a separate item: **align the experiment-harness windowing with the served windowing**, so a harness promotion actually predicts served behavior ? that mismatch is the real lesson here, beyond this one threshold.

Tell me which option you want (or "commit the doc") and I'll apply it on a branch + PR.

57. user (2026-06-14T10:54:22.141Z)

definitely commit the doc!

58. assistant (2026-06-14T10:54:52.743Z)

On it. Following the shared-checkout branch protocol (branch from `origin/master` explicitly, stage only the doc, re-verify HEAD before pushing). Let me fetch and check the remote first.

59. user (2026-06-14T10:54:56.235Z)

```
--- current branch ---
master
--- local HEAD ---
e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
--- origin/master HEAD ---
e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
--- did origin/master move / touch the doc? ---
--- working tree ---
 M docs/QUALITY_ITERATIONS.md
?? artifacts/
```

60. assistant (2026-06-14T10:55:01.061Z)

Clean: `origin/master == local HEAD == e75ec35`, no sibling commits, no remote changes to the doc. Branching from `origin/master` and committing the doc only.

61. user (2026-06-14T10:55:02.493Z)

```
M       docs/QUALITY_ITERATIONS.md
branch 'docs/served-gate-a-verification' set up to track 'origin/master'.
Switched to a new branch 'docs/served-gate-a-verification'
--- staged path (explicit) ---
M  docs/QUALITY_ITERATIONS.md
?? artifacts/
```

62. user (2026-06-14T10:55:24.807Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\commitmsg_servedgatea.txt (file state is current in your context ? no need to Read it back)

63. assistant (2026-06-14T10:55:18.906Z)

On the branch with only the doc staged (`artifacts/` untracked, not staged). Committing via a message file to avoid shell-quoting issues.

64. assistant (2026-06-14T10:55:24.795Z)

docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to production windowing

Owner-gated follow-up flagged in PR #146. Drove the live FusionSegmenter
served seams (_enrich_candidate_stats -> net_crossings, _select_for_handoff
-> the gate) over each golden video's CACHED WASB trajectory at the production
windowing process_video uses (vt=0.02, merge_gap=2.0, min_win=2.0). No WASB
re-run / GPU / Gemini.

```
(a) CONFIRMED: at min_crossings=0, fusion_hybrid seeding+handoff is
    byte-identical to wasb_hybrid on all 6 golden videos ? the no-regression
    invariant verified on live served code, not just by review.
(b) FINDING: the +0.074 win was measured on the harness's recall-oriented
    proposal windowing (over-segments, seg-ratio 1.68 -> 1.01). The coarse
    served windowing already merges aggressively, so Gate-A trims real
    rallies (net_crossings 0-1) instead of FPs. Candidate-handoff F1
    (IoU0.5, n=6): mc0 0.300 | mc1 0.303 (+0.003, 1 lost) |
    mc2 0.292 (-0.008, 12 lost) | mc3 0.269 (-0.031, 28 lost). Within n=6
    noise, but mc2 is not a served improvement and drops genuine rallies.

Root cause: eval-harness windowing != served windowing. Honest record only;
config.json untouched (mc=2 stays) ? the rollback/retune is owner-gated.
Repro: scratch/served_gateA_{verify,sweep}.py.

Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>
```

65. user (2026-06-14T10:55:30.968Z)

```
[docs/served-gate-a-verification f6acde5] docs(quality): served-side Gate-A check ? flip's
offline win doesn't transfer to production windowing
 1 file changed, 47 insertions(+)
--- re-verify: HEAD is mine, only my commit ahead of origin/master ---
docs/served-gate-a-verification
f6acde5 docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to
production windowing
--- files in the commit (must be ONLY the doc) ---
f6acde5 docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to
production windowing
 docs/QUALITY_ITERATIONS.md | 47 +++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 47 insertions(+)
```

66. user (2026-06-14T10:56:04.590Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\prbody_servedgatea.md (file state is current in your context ? no need to Read
it back)
```

67. assistant (2026-06-14T10:55:54.678Z)

```
Commit is clean: `f6acde5`, one file (`docs/QUALITY_ITERATIONS.md`, +47), only my commit ahead
of `origin/master`. Now pushing and opening a PR (docs-only, one work item).
```

68. assistant (2026-06-14T10:56:04.569Z)

Served-side Gate-A verification (follow-up to #146)

```
Owner-gated follow-up flagged by the Gate-A **SERVED-DEFAULT** entry in `QUALITY_ITERATIONS.md`.
Confirms whether the `min_crossings=2` default flip causes a served-side regression on the
**live** `FusionSegmenter` path ? driven over each golden video's **cached** WASB trajectory at
the production windowing `process_video` uses (`vt=0.02, merge_gap=2.0, min_win=2.0`, via
`load_config()`). No WASB re-run / GPU / Gemini.
```

(a) No-regression invariant ? CONFIRMED ?

```
At `min_crossings=0`, `fusion_hybrid` seeding **and** handoff are byte-identical to
`wasb_hybrid` on all 6 golden videos. The "fusion adds nothing when the gates are off" property
is now verified on the live served code, not just by #117's review.
```

(b) The +0.074 win does NOT transfer to the served windowing ??

```
The offline win was measured on the harness's recall-oriented **proposal** windowing
`(0.005,1.0,0.5)` where CONTROL over-segments (seg-ratio 1.68?1.01 = Gate-A cleaning
fragmentation). Production serves a **coarse** windowing that already merges aggressively, so
Gate-A has little FP to remove and instead trims **real rallies** (net_crossings 0?1).
```

Candidate-handoff F1 (pre-Gemini, IoU ? 0.5, n=6):

`min_crossings`	mean F1	precision	recall	real-rally windows lost	?F1
0 (off = `wasb_hybrid`)	0.300	0.317	0.290	0	?
1	**0.303**	0.327	0.288	1	**+0.003**
2 (shipped #146)	0.292	0.337	0.261	**12**	**?0.008**
3	0.269	0.338	0.226	28	?0.031

Within n=6 noise, but `mc=2` is **not** a served improvement and drops genuine rallies (worst: `largetest_doubles` ?0.051). A dropped candidate never reaches Gemini ? **unrecoverable served recall loss**, while the precision "gain" partly duplicates what Gemini already does.

Root cause: eval-harness windowing ? served windowing ? the harness promotes/refuses variants on a windowing production doesn't serve.

Scope

Docs-only. `config.json` is untouched (`mc=2` stays); the rollback/retune (recommended `mc=1`) is owner-gated. Repro: `scratch/served_gateA_{verify,sweep}.py` (gitignored); artifacts in `output/gateA_served_verify/`.

? Generated with [Claude Code](https://claude.com/claude-code)

69. user (2026-06-14T10:56:13.778Z)

```
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/docs/served-gate-a-verification
remote:
branch 'docs/served-gate-a-verification' set up to track 'origin/docs/served-gate-a-verification'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      docs/served-gate-a-verification -> docs/served-gate-a-verification
--- create PR ---
https://github.com/avidullu/badminton-highlight-indexer/pull/152
```

70. assistant (2026-06-14T10:56:25.476Z)

PR opened: [#152](https://github.com/avidullu/badminton-highlight-indexer/pull/152). Let me check CI status (gating explicitly on the status rollup, per the merge-discipline gotcha).

71. user (2026-06-14T10:56:47.787Z)

{"checks":[{"conclusion":"SUCCESS","name":"Ruff lint","status":"COMPLETED"},{"conclusion":"","name":"Mypy (lenient baseline)","status":"IN_PROGRESS"},{"conclusion":"SUCCESS","name":"Syntax sweep + import smoke (no GPU stack)","status":"COMPLETED"},{"conclusion":"","name":"Full test suite (offline, mocked)","status":"IN_PROGRESS"}],"mergeStateStatus":"UNKNOWN","mergeable":"UNKNOWN"}

72. assistant (2026-06-14T10:56:57.989Z)

CI running (2 of 4 green, mypy + full suite in progress). Let me wait for it to finish.

73. user (2026-06-14T10:56:58.734Z)

<tool_use_error>Blocked: sleep 110 followed by: gh pr view 152 --json statusCheckRollup --jq '[.statusCheckRollup[]|{name,status,conclusion}]' head -40. To wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use run_in_background: true. Do not chain shorter sleeps to work around this block.</tool_use_error>

74. user (2026-06-14T10:57:05.016Z)

{"checks":[{"conclusion":"SUCCESS","name":"Ruff lint"},{"conclusion":"SUCCESS","name":"Mypy (lenient baseline)"},{"conclusion":"SUCCESS","name":"Syntax sweep + import smoke (no GPU

stack)"},{"conclusion":"","name":"Full test suite (offline,
mocked)"}],"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}

75. assistant (2026-06-14T10:57:15.996Z)

3 of 4 green; full test suite still running (`UNSTABLE` = pending check). I won't merge until all are SUCCESS. Let me wait on it with an until-loop.

76. user (2026-06-14T10:58:29.242Z)

waited ~60s
{"checks":[{"conclusion":"SUCCESS","name":"Ruff lint"},{"conclusion":"SUCCESS","name":"Mypy
(lenient baseline)"},{"conclusion":"SUCCESS","name":"Syntax sweep + import smoke (no GPU
stack)"},{"conclusion":"SUCCESS","name":"Full test suite (offline,
mocked)"}],"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}

77. assistant (2026-06-14T10:58:37.628Z)

All 4 CI lanes green, `mergeStateStatus: CLEAN`. This is a docs-only honest record ? within my standing authority to merge on verified-green CI, and you clearly want it landed. Merging (squash) and cleaning up.

78. user (2026-06-14T10:58:47.361Z)

```
 tests/test_fusion_hybrid.py                |  86 +------------
 tests/test_served_gate_a_regression.py     | 160 ++++++++++++++++++++++
 8 files changed, 478 insertions(+), 157 deletions(-)
 create mode 100644 backend/eval/served_gate_a.py
 create mode 100644 tests/test_served_gate_a_regression.py
--- back to master, ff pull ---
Your branch is up to date with 'origin/master'.
Already on 'master'
Already up to date.
--- verify landed + clean ---
53bc743 docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to
production windowing (#152)
2cd469e revert(fusion): undo the Gate-A served-default flip (#146) ? confirmed served regression
(eval?serve) (#151)
?? artifacts/
--- tidy temp commit/PR message files ---
removed temp msg files (repro scripts kept)
```

79. assistant (2026-06-14T10:59:21.313Z)

Important: while my docs PR was in flight, a **sibling session merged [#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151)** ? a revert of the Gate-A flip that independently reached the same conclusion ("confirmed served regression (eval?serve)") and even added a `backend/eval/served_gate_a.py` module + regression test. My docs PR #152 landed cleanly on top (`53bc743`, CI green). Let me reconcile ? my doc entry currently presents the rollback as still-pending, which may now be stale. Let me see what #151 actually did.

80. user (2026-06-14T10:59:24.083Z)

```
=== config.json now (did #151 flip mc back?) ===
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import json;f=json.load(open('config.json'))['indexing'];print('default_segmenter
=',f['default_segmenter']);print('fusion.min_crossings =',f['fusion']['min_crossings'])
                                                ~^^^^^^^^^^
KeyError: 'fusion'
default_segmenter = gemini

=== #151 commit body ===
```

revert(fusion): undo the Gate-A served-default flip (#146) ? confirmed served regression (eval?serve) (#151)

Restore the pre-#146 no-regression production state. The #146 flip promoted Gate A (net-crossings >= 2) from a recommended config to the live served default, on the strength of an offline +0.074 ?F1. A served-side check (the #148 litmus + an independent served run) found the lift does NOT transfer: on the production COARSE windowing Gate A is neutral-to-negative (mean served ?F1 -0.008, ~12 real rallies dropped, 2/6 losses).

Root cause (eval?serve, not a bug): the +0.074 is a property of the offline harness's permissive PROPOSAL windowing (vt=0.005, gap=1.0, min=0.5), which emits many short windows with held-shuttle junk for Gate A to cut. The served COARSE windowing (vt=0.02, gap=2.0, min=2.0) already under-segments (over-seg ratio < 1), so there is little junk left and the gate trims borderline true windows instead.

Reverted (back to the pre-#146 state):
- config.json: indexing.default_segmenter fusion_hybrid -> gemini (the prior served detector); the #146-added indexing.fusion block removed.
- backend/config/models.py: FusionConfig.min_crossings default 2 -> 0 (Gate A OFF by default). Gate A is NOT deleted ? it stays opt-in (indexing.fusion.min_crossings: 2).
- tests/test_config.py, tests/test_fusion_hybrid.py: restored to the Gate-A-OFF defaults (test_fusion_hybrid.py is byte-identical to its pre-#146 form). The persisted-superset invariant is preserved ? gating only ever filtered the AI handoff, never persistence.
- docs/QUALITY_ITERATIONS.md: the Gate-A SERVED-DEFAULT entry now reads REVERTED with the honest eval?serve reason; the PROMOTED entry's stale "owner approved the flip" pointer is corrected.
- eval_baselines/experiment_leaderboard.json: the #146 promotion record carries a "reverted (eval?serve)" note.

The #148 served litmus (backend/eval/served_gate_a.py + tests/test_served_gate_a_regression.py) is the standing guard that caught this and pins the honest measured behaviour.

ruff + mypy clean; full suite 605 passed, 2 skipped (golden-data-gated).

Co-authored-by: Claude Opus 4.8 (1M context) <noreply@anthropic.com>

```
=== #151 files changed ===
2cd469e revert(fusion): undo the Gate-A served-default flip (#146) ? confirmed served regression
(eval?serve) (#151)
 backend/config/models.py                   | 33 +++---------
 config.json                                |  8 +--
 docs/QUALITY_ITERATIONS.md                 | 63 +++++++++++++---------
 eval_baselines/experiment_leaderboard.json |  1 +
 tests/test_config.py                       | 38 +++++++------
 tests/test_fusion_hybrid.py                | 86 ++----------------------------
 6 files changed, 72 insertions(+), 157 deletions(-)
```

81. assistant (2026-06-14T11:00:06.634Z)

Now I have the full picture. A sibling session did the complete rollback in parallel:
- **PR #148** productionized a served-side litmus: `backend/eval/served_gate_a.py` + `tests/test_served_gate_a_regression.py` (standing guard).
- **PR #151** reverted #146: `config.json` ? `default_segmenter: gemini` (the `indexing.fusion` block removed ? that's why my probe hit `KeyError: 'fusion'`), `FusionConfig.min_crossings` default 2?0 (Gate A now opt-in), and it already updated the SERVED-DEFAULT doc entry to "REVERTED". Its independent numbers **match mine** (?0.008, ~12 rallies dropped, 2/6 losses).

So my just-merged entry is now **stale** ? it says "config.json untouched (mc=2 stays)" and frames the rollback as pending, when it's already done. That misstates the live state. I need to reconcile the doc to keep it honest. Let me read the current top sections to fix it precisely without contradicting #151's wording.

82. user (2026-06-14T11:00:07.755Z)

43   ### Served-side Gate-A check on the LIVE FusionSegmenter ? the flip's offline win does

NOT transfer to the production windowing *(2026-06-14)*

The owner-gated follow-up flagged by the SERVED-DEFAULT entry below: confirm the live `FusionSegmenter` served path has no regression from the Gate-A default flip (PR #146). Driven the **real** served seams (`_enrich_candidate_stats` ? `net_crossings`, `_select_for_handoff` ? the gate) over each golden video's **cached** WASB trajectory, at the **production windowing** `process_video` actually uses (`velocity_thresh=0.02, merge_gap=2.0, min_window=2.0` from config) ? no WASB re-run (the runner has no cache-hit early-return), no GPU, no Gemini. Config loaded exactly as production (`load_config()`); segmenters from the registry. Repro: `scratch/served_gateA_verify.py` + `scratch/served_gateA_sweep.py`; artifacts isolated in `output/gateA_served_verify/`.

- **(a) No-regression invariant CONFIRMED on real served code.** At `min_crossings = 0`, `fusion_hybrid`'s seeding **and** handoff set are **byte-identical to `wasb_hybrid`** on all 6 golden videos (same candidate windows; handoff = all windows = the base seam). The "fusion adds nothing when the gates are off" property is now verified on the *live* served path, not only by code review. ?
- **(b) The +0.074 win does NOT transfer to the served windowing.** The offline promotion (+0.074, 6/6, p=0.031) was measured on the eval harness's **recall-oriented *proposal* windowing** `(0.005, 1.0, 0.5)`, where CONTROL **over-segments** (segment-count ratio 1.68) and Gate-A's job is to clean up that fragmentation (? 1.01). The **served path uses a coarse windowing** `(0.02, 2.0, 2.0)` that already merges aggressively (low over-segmentation), so Gate-A has little FP to remove and instead trims **real** rallies whose WASB trajectory registered only 0?1 net crossings. **Candidate-handoff-level F1 (pre-Gemini, IoU ? 0.5) on the served windowing, n=6:**

min_crossings	mean F1	precision	recall	real-rally windows lost vs off	?F1 vs off
0 (Gate-A OFF = `wasb_hybrid`)	0.300	0.317	0.290	0	?
1	**0.303**	0.327	0.288	1	**+0.003**
2 (shipped default, PR #146)	0.292	0.337	0.261	**12**	**?0.008**
3	0.269	0.338	0.226	28	?0.031

  At the shipped `mc=2`, mean F1 goes **0.300 ? 0.292 (?0.008)** and Gate-A drops **12 windows that match golden rallies** (net_crossings almost all exactly 1.0), concentrated in doubles (`largetest_doubles` ?0.051). All deltas are **tiny / within n=6 noise**, so this is not a catastrophic regression ? but `mc=2` is **not** a served-side *improvement*, and the held-shuttle "guard" is removing genuine rallies, not just feeds.
- **Served relevance (honest).** A dropped candidate window **never reaches Gemini**, so each lost real rally is an **unrecoverable served recall loss**; the precision "gain", by contrast, is partly what Gemini already does at the handoff ? so the served-side tradeoff is at best neutral and plausibly net-negative, i.e. the candidate-level ?0.008 likely *understates* the served downside.
- **Root cause = eval ? serve windowing.** The harness promotes/refuses variants on a windowing the production path does not serve. Gate-A's value is conditional on over-segmentation; the coarse served windowing doesn't over-segment, so the gate inverts. [[decision-rigor-norm]]: the foundational issue (harness windowing ? served windowing) outweighs the single proposal-windowing number.

Recommendation (owner-gated, single-flag). `mc=1` is the better served operating point if Gate-A stays on (?flat F1, only 1 real rally lost, +0.010 precision); `mc=0` is the clean rollback. Either is a one-line config change (no code): `indexing.fusion.min_crossings: 1` or `: 0` (or full revert `indexing.default_segmenter: wasb_hybrid`). The deeper fix is to **align the experiment-harness windowing with the served windowing** (or re-run promotions on the production windowing) before treating a harness lift as a served default. The offline ablation and unit tests remain valid for what they measured; this adds the served-side number the flip entry said was still owed.

PERSONALIZATION Phase-0 quality checkpoint ? the eval/safety RAILS for the per-user feature *(2026-06-14)*

The owner adopted the **HYBRID** posture: personalization as a **flag-gated FEATURE on a generalization-first baseline**, paired with a **contribution flywheel** so per-user gains and the shared baseline compound together (full design in PERSONALIZATION_PLAN.md, merged [#144](https://github.com/avidullu/badminton-highlight-indexer/pull/144)). This checkpoint landed the three **honest-eval RAILS** that keep a per-user/small-N tuning layer from promoting noise or silently regressing a user:

- **Per-user promotion gate ? [#141](https://github.com/avidullu/badminton-highlight-indexer/pull/141)** (`backend/eval/per_user_gate.py`): a **min-N floor** + **structural-guard exemption** ? per-user/small-N tuning can't promote noise and can't disable a structural guard (the proven Gate-A held-shuttle guard stays on regardless of a thin user corpus).
- **Held-out-USER learning curve ? [#142](https://github.com/avidullu/badminton-highlight-indexer/pull/142)** (`backend/eval/per_user_eval.py`): turns the vague "<K videos to superior quality" claim into a **falsifiable release number** (the `smallest_k_with_lift` measured on held-out users).
- **Per-user no-regression guard ? [#143](https://github.com/avidullu/badminton-highlight-indexer/pull/143)**: a retrain/baseline-upgrade can **never make a user worse than their incumbent** (frozen per-user CONTROL; a regressing candidate is refused).

Read HONESTLY: these are the **eval/safety rails** for the personalization feature ? **not** new measured rally-detection wins. No F1 moved this checkpoint; what landed is the discipline that lets per-user tuning ship without regressing the baseline or over-claiming on n?6-per-user data.

Locked owner decisions (2026-06-14): identity = *design for the north-star (multi-user/cloud), implement for current use (single-user local)*; the inline per-user classifier sits **behind a cloud-load interface** (local store = one impl); **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate** (a one-video user is ALWAYS fully segmented by the baseline batteries); **free-tier videos ARE pooled** into the shared baseline (consent baked into free terms, minors-exclusion enforced at the pooling query); and the previously-pending structural-guard sub-decision is now LOCKED ? **Gate-A-only structural + `min_n = 5`**.

Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising (not promotable) *(2026-06-14)*

The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ? `output/*_golden_features.json`, the replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats (C1 ? bootstrap 95% CI + paired sign

122 test; never a bare mean at n=6)**:
123
124 | Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears 0 |
125 |---|---|---|---|---|---|
126 | shuttle-only | 0.307 | ? | ? | ? | ? |
127 | + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) | **No** |
128 | + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L (p=0.688) | **No** |
129
130 **Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay
131 default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ? `players_in_court` counts spectators);
132 per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead ? +0.079 / +27.7%, wins 4/6
133 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above** shuttle-only (0.307) ? but n=6 can't
134 confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and measure audio on shuttle-only
135 *directly* (without the person handicap). Recorded in `eval_baselines/experiment_leaderboard.json`. **Negative/uncertain
136 results are kept on purpose ? this is the honest ablation backbone for the paper aim.**
137
138 ### Gate-A served-default flip ? **REVERTED (eval?serve regression)** *(2026-06-14)*
139 **The #146 flip of Gate-A (net-crossings ? 2) to the live served default was REVERTED.** A served-side check ?
140 the repeatable litmus in [#148](https://github.com/avidullu/badminton-highlight-indexer/pull/148)
141 (`backend/eval/served_gate_a.py` + `tests/test_served_gate_a_regression.py`), corroborated by an independent
142 served run ? found Gate A is **not a win on the served path**: on the production (COARSE) windowing it is

83. assistant (2026-06-14T11:01:03.752Z)

Now I understand the full state: the doc now has my entry (stale ? says "mc=2 stays / decision pending") AND #151's authoritative "REVERTED" entry below it. My entry misstates the live state. I'll reconcile it to reflect reality (flip already reverted by #151, corroborated by my independent run) ? keeping its unique value (the byte-identity (a) confirmation + the mc-sweep) but fixing the stale framing.

84. user (2026-06-14T11:01:11.605Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md has been updated successfully. (file state is current in your context ? no need to Read it back)

85. user (2026-06-14T11:01:22.296Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md has been updated successfully. (file state is current in your context ? no need to Read it back)

86. assistant (2026-06-14T11:01:40.142Z)

Now let me fix the memory checkpoint ? it also has the stale "decision pending / not committed yet" framing.

87. user (2026-06-14T11:01:49.684Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

88. assistant (2026-06-14T11:01:54.567Z)

Memory updated. Now commit the doc reconciliation ? fetching first since sibling sessions are clearly active.

89. user (2026-06-14T11:01:57.656Z)

```
--- local vs origin/master ---
53bc743 docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to
production windowing (#152)
53bc743 docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to
production windowing (#152)
--- branch from origin/master ---
branch 'docs/served-gate-a-reconcile' set up to track 'origin/master'.
Switched to a new branch 'docs/served-gate-a-reconcile'
M  docs/QUALITY_ITERATIONS.md
?? artifacts/
```

90. assistant (2026-06-14T11:02:08.289Z)

```
docs(quality): reconcile served-side Gate-A entry with the #151 revert

My #152 entry was authored while the rollback decision looked pending; a sibling
session had already (a) productionized the served litmus in #148
(backend/eval/served_gate_a.py + tests/test_served_gate_a_regression.py) and
(b) REVERTED the #146 flip in #151 (config default_segmenter -> gemini, the
indexing.fusion block removed, FusionConfig.min_crossings default 2 -> 0 so Gate A
is OFF-by-default-but-opt-in).

This corrects the now-stale framing so the log stays honest:
- header/intro reframed as the INDEPENDENT measurement that corroborates #148/#151
  (our numbers match: served ?F1 -0.008, ~12 rallies dropped, 2/6 losses);
- the "Recommendation (decision pending)" paragraph -> "Outcome: REVERTED in #151"
  with the actual config state and a cross-link to the REVERTED entry;
- points at the productionized guard backend/eval/served_gate_a.py (#148), not only
  the scratch repros.

Measurement, tables, and the (a) mc=0 byte-identity-to-wasb_hybrid confirmation are
unchanged. Open follow-up unchanged: align the harness windowing with the served
windowing before treating a harness lift as a served default.

Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>
```

91. user (2026-06-14T11:02:08.453Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\commitmsg_reconcile.txt (file state is current in your context ? no need to Read it back)

92. user (2026-06-14T11:02:24.863Z)

```
[docs/served-gate-a-reconcile 869c870] docs(quality): reconcile served-side Gate-A entry with
the #151 revert
 1 file changed, 19 insertions(+), 14 deletions(-)
--- verify only my commit + only the doc ---
869c870 docs(quality): reconcile served-side Gate-A entry with the #151 revert
869c870 docs(quality): reconcile served-side Gate-A entry with the #151 revert
 docs/QUALITY_ITERATIONS.md | 33 +++++++++++++++++++-------------------
 1 file changed, 19 insertions(+), 14 deletions(-)
--- push + PR ---
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      docs/served-gate-a-reconcile -> docs/served-gate-a-reconcile
https://github.com/avidullu/badminton-highlight-indexer/pull/153
```

93. assistant (2026-06-14T11:02:30.074Z)

PR [#153](https://github.com/avidullu/badminton-highlight-indexer/pull/153) opened (docs-only, 1 file). Let me wait for CI and merge on green.

94. user (2026-06-14T11:04:33.371Z)

Command running in background with ID: bc7j0nc84. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\e03c8ada-c18d-40e6-9ee1-144846fe3975\tasks\bc7j0nc84.output. You will be notified when it completes. To check interim output, use Read on that file path.

95. assistant (2026-06-14T11:04:50.089Z)

CI for the reconcile PR is running in the background; I'll merge it once it goes green. Quick status while that finishes:

What happened: Your "commit the doc" landed as [PR #152](https://github.com/avidullu/badminton-highlight-indexer/pull/152) (merged, `53bc743`, CI green). But I then discovered a **sibling session had already acted on the same finding in parallel**:
- **[#148](https://github.com/avidullu/badminton-highlight-indexer/pull/148)** productionized a served-side litmus (`backend/eval/served_gate_a.py` + a regression test).
- **[#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151)** **reverted the #146 flip** ? `config.json` `default_segmenter` ? `gemini`, the `indexing.fusion` block removed, `FusionConfig.min_crossings` default `2?0` (Gate A now OFF-by-default but still opt-in).

Their independent numbers **match mine** (served ?F1 ?0.008, ~12 rallies dropped, 2/6 losses) ? good corroboration that the eval?serve regression is real.

That made my freshly-merged #152 entry **stale** (it framed the rollback as "decision pending / mc=2 stays"). So [PR #153](https://github.com/avidullu/badminton-highlight-indexer/pull/153) reconciles it: reframed as the independent measurement corroborating #148/#151, "Outcome ? REVERTED" instead of "decision pending", pointing at the productionized guard. Memory checkpoint updated to match.

Net live state: the Gate-A served-default flip is **already reverted** ? your original question (rollback to mc=0 / retune to mc=1 / leave mc=2) is effectively moot; production is back on the pre-#146 `gemini` default with Gate A opt-in. The one genuinely open item is the deeper lesson: **align the experiment-harness windowing with the served windowing** so a harness promotion actually predicts served behavior.

I'll confirm #153 merged once CI completes.

96. user (2026-06-14T11:04:50.173Z)

<task-notification>
<task-id>bc7j0nc84</task-id>
<tool-use-id>toolu_01XFNrchpiwuHWDCfpJ1cj7q</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\e03c8ada-c18d-40e6-9ee1-144846fe3975\tasks\bc7j0nc84.output</output-file>
<status>completed</status>
<summary>Background command "Poll until all CI checks conclude on PR #153" completed (exit code 0)</summary>
</task-notification>

97. user (2026-06-14T11:05:00.747Z)

{"checks":[{"conclusion":"SUCCESS","name":"Ruff lint"},{"conclusion":"SUCCESS","name":"Mypy (lenient baseline)"},{"conclusion":"SUCCESS","name":"Syntax sweep + import smoke (no GPU stack)"},{"conclusion":"SUCCESS","name":"Full test suite (offline, mocked)"}],"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}

98. assistant (2026-06-14T11:05:03.799Z)

All 4 lanes green, `CLEAN`. Merging the reconcile PR and returning to a clean master.

99. user (2026-06-14T11:05:12.675Z)

```
Fast-forward
 docs/QUALITY_ITERATIONS.md | 33 +++++++++++++++++++--------------
 1 file changed, 19 insertions(+), 14 deletions(-)
Already on 'master'
Already up to date.
--- final state ---
a03f0f4 docs(quality): reconcile served-side Gate-A entry with the #151 revert (#153)
53bc743 docs(quality): served-side Gate-A check ? flip's offline win doesn't transfer to
production windowing (#152)
2cd469e revert(fusion): undo the Gate-A served-default flip (#146) ? confirmed served regression
(eval?serve) (#151)
?? artifacts/
--- live config sanity (post-#151 revert) ---
default_segmenter = gemini | fusion block present: False
```

100. assistant (2026-06-14T11:05:35.565Z)

Done ? both PRs merged, master clean at `a03f0f4`, live config confirmed back to the pre-#146 state (`default_segmenter: gemini`, no `fusion` block, Gate A off-by-default).

Final outcome

Served-side verification (the task):
- **(a) No-regression invariant ? CONFIRMED ?** At `min_crossings=0`, the live `FusionSegmenter` seeding + handoff are byte-identical to `wasb_hybrid` on all 6 golden videos ? verified on real served code, not just by review.
- **(b) Regression found ??** The offline `+0.074` win does **not** transfer to the production (coarse) windowing. At the shipped `mc=2`: mean F1 ?0.008, **12 real-rally windows dropped** (worst on doubles). Root cause: **eval-harness windowing ? served windowing** ? Gate-A's value was an artifact of the over-segmenting *proposal* windowing the harness uses but production doesn't serve.

What landed:
- [#152](https://github.com/avidullu/badminton-highlight-indexer/pull/152) ? the honest measurement recorded in `docs/QUALITY_ITERATIONS.md` (your "commit the doc").
- [#153](https://github.com/avidullu/badminton-highlight-indexer/pull/153) ? reconciled that entry with reality, since a sibling session acted on the same finding in parallel.

The parallel-session twist (worth knowing): while I was verifying, other slates independently reached the same conclusion and went further ? **[#148](https://github.com/avidullu/badminton-highlight-indexer/pull/148)** productionized a served-side litmus (`backend/eval/served_gate_a.py` + regression test) and **[#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151)** reverted the flip. Their numbers match mine (?0.008, ~12 dropped, 2/6 losses) ? independent corroboration. So the rollback decision I queued for you is **already executed**; nothing for you to decide there.

One genuinely open item (the real lesson, not requested ? flagging only): the experiment harness promotes/refuses cues on a windowing production doesn't serve, so a harness lift doesn't reliably predict served behavior. Aligning the harness windowing with the served windowing (or re-running promotions on production windowing) would prevent the next eval?serve surprise. I can scope that as its own work item whenever you want ? it's the kind of thing worth a proper design pass.